

2. SEMESTER EKSAMENSRAPPORT

Smilfotograferne

Anslag: 63.511

Dato: 28/05/2026

Github: <https://github.com/lukasbjohansen/2AGruppeLKMF>

Hjemmeside: <http://skolefotograferne26.kasperdat.dk/>

Video: <https://youtu.be/oPScIQg-48I>

Udarbejdet af
Kasper K. Møller,
Maria L. Strauss,
Frederik H.
Kristensen, Sinan
Faisal og Lukas B.
Johansen

Indholdsfortegnelse

Indledning (Maria og Kasper)	4
Virksomhedsanalyse	5
Business Model Generation Canvas (BMGC) (Kasper, Frederik, Maria og Lukas)	5
1. Kundesegmenter	5
2. Værditilbud	5
3. Kanaler (fælles)	6
4. Kunderelationer	7
5. Indtægtskilder (fælles)	7
6. Nøgleressourcer (fælles)	7
7. Nøgleaktiviteter (fælles)	8
8. Nøglepartnere (Kasper og Frederik)	9
9. Omkostningsstruktur	10
SWOT-Analyse (fælles)	11
Inception Deck (Projekt-"Pitch") (fælles)	12
1. <i>Why are we here?</i> (Hvad er det brændende problem?)	12
2. Elevator pitch	12
3. The Product Box (Visualisering af produktæskan)	13
4. <i>The "Not-List"</i> (Hvad laver vi <i>ikke</i> i dette projekt? Vigtigt for afgrænsning) (Frederik)	14
5. <i>Meet the Neighbors</i> (Hvem er interessenterne? F.eks. bestyrelsen, menige medlemmer, kassereren) (Maria)	15
6. Show The Solution (En overordnet beskrivelse af det system, I vil bygge) (Maria)	16
7. What Keeps Us Up At Night (Risiko analyse over projekts udvikling) (Kasper, Sinan, Frederik og Maria)	17
8. Size it up (fælles)	18
9. Be clear on what's going to give (Kasper, Maria og Frederik)	19
10. Show what it's going to take	20
Analyse, design og implementering	21
Navneordsteknikken og tanker om domænet (Lukas, Frederik, Kasper og Maria)	21
Domænemodel (Frederik, Lukas, Kasper og Maria)	23

Overvejelser og tanker om domænemodel, konceptuelle klasser og relationer (Frederik, Lukas, Kasper og Maria):	24
ER-diagram (Lukas, Frederik, Kasper og Maria)	26
Klassediagram (diskuteret og udarbejdet fælles)	27
Første udkast (skrevet af Lukas)	27
Projektopbygning – mappestruktur i Solution Explorer (Frederik)	29
Det endelige klassediagram – uden relationer (Kasper, Sinan og Maria)	30
Det endelige klassediagram – med relationer (udarbejdet af Kasper og Maria, diskuteret i fællesskab og beskrevet af Lukas og Sinan)	34
Userstories (udarbejdet fælles, skrevet af Maria).....	36
Burndown Chart (Kasper)	37
Estimering af userstories (Frederik, Lukas, Kasper og Maria)	38
GUI – prototype og tanker om front-end (udarbejdet i fællesskab, beskrevet af Frederik, Sinan og Maria)	39
Mockup prototype (udarbejdet i fællesskab, beskrevet af Maria, Frederik og Kasper)	42
Udviklingsproces	46
Scrum (Maria)	46
Sprint 0 - uge 12-18	46
Sprint Planning (16/03/2026) (fælles)	46
Sprint review (01/05/2026) (Frederik, Kasper og Maria)	47
Sprint retrospective (01/05/2026) (Frederik, Kasper og Maria)	48
Sprint 1 - uge 19.....	49
Sprint Planning (01/05/2026) (fælles)	49
Sprint backlog og arbejdsproces (Maria)	51
Kodeeksempler	52
Sprint review (08/05/2026) (fælles)	56
Sprint retrospective (08/05/2026)(fælles)	58
Sprint 2 - uge 20.....	60
Sprint Planning (08/05/2026) (fælles)	60
Sprint arbejdsproces (Maria)	60
Kodeeksempler	62
Sprint review (15/05/2026) (Maria og Kasper).....	65

Sprint retrospective (15/05/2026) (fælles)	68
Sprint 3 - uge 21.....	69
Sprint Planning (15/05/2026) (fælles)	69
Sprint backlog og arbejdsproces (Kasper, Frederik og Maria)	70
Kodeeksempler:.....	71
Sprint Review (22/05/2026) (Maria)	74
Sprint 4 - uge 22.....	76
Sprint Planning (22/05/2026) (fælles)	76
Sprint Review (27/05/2026) (Frederik, Maria, Kasper og Lukas)	77
Sprint Retrospective	78
Refleksion (diskuteret i fællesskab, skrevet af Maria)	78
Arbejdsproces.....	78
Samarbejde	79
Konklusion (Maria, Kasper og Frederik)	80
Bibliografi	81
Links	82
Bilag.....	83
Bilag 1 – Daily Scrums	83
Bilag 2 – Elevator Pitch skabelon.....	118
Bilag 3.....	119
Bilag 4 – Scrum Event planlægning.....	119
Scrum Events	119

Indledning (Maria og Kasper)

Skolefotograferne er en skolefotograferingsvirksomhed, som tilbyder forældre og elever en ny tilgang til det traditionelle skolefoto koncept.

Det nye i Vicki's koncept er, at forældrene selv beslutter om de ønsker billederne, hvor mange, de vil købe og i hvilken størrelse. Alt dette foregår online gennem en hjemmeside, der forener skolesekretærer, skoleledere, fotografer og forældre og gør bestillings- og købsprocessen lettilgængelig, fleksibel og overskuelig.

Hendes nuværende system er dog fyldt med mangler, som gjorde hendes hjemmeside uholdbar for hende og hendes kunder.¹

Skolefotograferne har derfor kontaktet og hyret os til at designe et nyt og opdateret system.

Problemformulering:

"Hvordan kan den agile udviklingsmetode, C#-Programmering og Relational Databaser benyttes til at udvikle, designe og implementere et mindre IT system?"

¹ (Skolefoto-servicesystem 2026, 2026)

Virksomhedsanalyse

Business Model Generation Canvas (BMGC) (Kasper, Frederik, Maria og Lukas)

Vi har brugt Business Model Generation Canvas² forretningsmodellen til at undersøge, hvordan Skolefotograferne hhv. skaber, leverer og tilegner sig værdi. Dette for at sikre os en fælles forståelse for forretningen for bedre at kunne imødekomme product owners ønsker og behov.

1. Kundesegmenter

1.1 Nichemarked (Frederik)

Skolefotograferne er et nichemarked, da kundesegmentet består af og henvender sig til forældre. Og ikke et massemarked, som fx MitID, der henvender sig til alle.

1.2 Roller/brugergrupper: (Fælles)

- Forældre
- Skolesekretærer
- Fotografer
- Lærere
- Systemadministrator (Vicki)

2. Værditilbud

2.1 Newness (Maria)

Behov for billeder, som har en fortælling og ikke bare er et blankt, opstillet og upersonligt portræt. Skolefotograferne vil skabe en personlig og skræddersyet fotooplevelse, og få den/de portrætteredes personlighed med i billederne.

² (Osterwalder & Pigneur, 2009)

De fokuserer ligeledes på at skabe en mere miljøvenlig og bæredygtig fotoløsning, hvor de søger at digitalisere fotos og gøre det muligt for forældre at vælge præcis de billeder og størrelser, de ønsker, frem for fysisk masse-print af fotobøger og billeder i diverse ubrugelige størrelser, som ofte ender i skraldespanden.

2.2 Arbejdsudbud (fælles)

Selvom fotograferne ikke er en decideret kunde, skaber Skolefotograferne værdi for fotograferne ved at give dem arbejdskraft og promovere dem.

2.3 Mulighed for tilpasning (fælles)

Forældrene har mulighed for at tilpasse deres ordre til deres behov, fx billedstørrelse, antal billeder, farve/sort-hvid m.m.

2.4 Fleksibilitet (Lukas)

Skolefotograferne skaber desuden fleksibilitet for brugeren, da de har mulighed for at booke tid uden for skoletiden.

3. Kanaler (fælles)

3.1 Direkte kanal

Websalg via hjemmesiden. På hjemmesiden kan Skolefotograferne promovere sig selv til skoler, fotografer og forældre.

Derudover er e-mails også en vigtig kanal, da det giver Vicki direkte kontakt til forældrene. Det er samtidig gennem e-mails, at leveringen af produkterne foregår samt kundekontakt- og support.

3.2 Indirekte kanal

Vicki promoverer sig selv ved fysisk at møde op på skolerne og sælge sit produkt.

4. Kunderelationer

4.1 Automatiseret service (Frederik)

Kunden oplever en personaliseret, men automatisk kontakt gennem hjemmesiden.

4.2 Selvbetjening (Frederik)

Kunden klarer selv hele processen, uden menneskekontakt, gennem hjemmesiden.

4.3 Dedikeret personlig assistance (Maria)

Man kan også argumentere for, at skolefotograferne etablerer et 'dedicated personal assistance' forhold til eleverne, i og med fotograferne skaber en personlig fotooplevelse med eleverne én-til-én. Dog er eleverne ikke som sådan 'kunderne' i relationen, da kunderne er forældrene, men eleverne indgår dog stadig i kunderelationen.

5. Indtægtskilder (fælles)

5.1 Salg af aktiver

Forældrene betaler for en professionel, men også nytænkende, skræddersyet og fleksibel fotooplevelse af deres børn. Fotograferingen omhandler både det traditionelle klassebillede på skolen, men også portrætter, både i og udenfor skoletiden og med/uden søskende fra samme eller omkringliggende skoler.

6. Nøgleressourcer (fælles)

6.1 Menneskelige

- Fotografer
- Ejeren af Skolefotograferne (Vicki)
- Relation til mange skoler og godt omdømme (Vicki)

6.2 Fysiske

- Skolerne stiller lokaler til rådighed (efter skoletid)
- Kameraer/fotografernes eget udstyr
- Fotopapir
- Fotolaboratorie

6.3 Intellektuelle

- Erfaring indenfor skolefotografering
- Salgserfaring

7. Nøgleaktiviteter (fælles)

7.1 Fotografering

- Fotografering af elever, personale og evt. familie

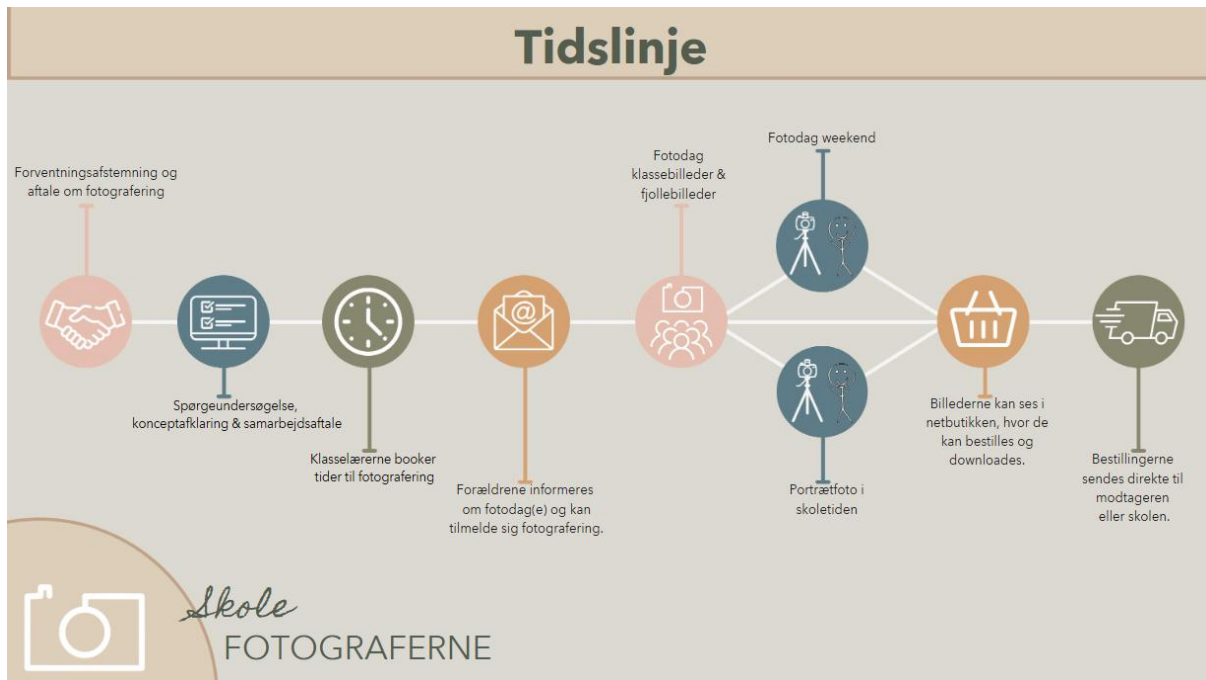
7.2 Produktion

- Fotoprint til forældre og skoler

7.3 Service

- Skoler har tradition for skolefoto og skal bruge nogen til at tilbyde den service
- Fotostudie for forældre på skolens areal

Her ses tidslinjen over købsprocessen, fra aftalen mellem skole og fotograf falder på plads, til forældrene står med de fysiske billeder i hånden.



Vicki's tidslinje over købsprocessen (Maddison, 2023).

8. Nøglepartnere (Kasper og Frederik)

- Folkeskoler
- Freelance selvstændige fotografer
- Investor (sparringspartner) ift. opbygning af virksomhed og hjemmeside. Hjælp også med at finde det udenlandske udviklerteam
- Hosting til billeduploads og hjemmeside
- Virksomhed til fotoprint - nuværende *billedprint laboratorie* ligger i Portugal, men fra efteråret 2026 forventer hun at skifte til *Nordic Digital Lab*, beliggende i Danmark
- DAO (til levering af billeder privat til forældrene)

9. Omkostningsstruktur

9.1 Faste omkostninger (Lukas)

- Webhosting, serveromkostninger og domæne
- Husleje til fotostudie (til private sessions)
- Forsikringer

9.2 Variable omkostninger (Kasper og Frederik)

- Fotoprint
- Lønudgifter til Vicki og eventuelt andre fotografer (50-65% af indtægterne fra forældrene går til fotografernes løn)
- Udgifter til Vickis udstyr (indkøb og vedligeholdelse af kamera, lys etc.). De eksterne fotografer medbringer og betaler selv deres eget udstyr
- Aflønning af webudviklere

9.3 Omkostningsdrevet (Maria)

- *Skolefotograferne* henvender sig til den 'almindelige' forældre og fokuserer på at tilbyde høj kvalitets billeder og service, men til konkurrencedygtige priser

9.4 Værdidrevet (Maria)

- Selvom *Skolefotograferne* ikke er et dyrt luksusbrand, fokuserer de på en personaliseret fotooplevelse, som giver forældrene høj værdi for pengene

SWOT-Analyse (fælles)

Vi har brugt SWOT³ analysen til at analysere virksomhedens interne styrker og svagheder, og eksterne muligheder og trusler.

	Styrker	Svagheder
Interne	• Skræddersyet foto oplevelse • Professionelle fotografer • Bæredygtighed (digitale fotos frem for fysisk print) • Fleksibilitet (fotografering uden for skoletiden) • Godt omdømme i branchen (Vicki er en eftertragtet og erfaren skolefotograf)	• Ikke-brugervenlig hjemmeside • Booking- og leveringsfejl • Manglende kommunikation (forældre og Vicki imellem) • Sikkerhedsrisici (GDPR) • Afgrænset kundesegment • Datatab og fejl i systemet, når fotografer manuelt skal indtaste billedstier • Tidskrævende og ikke særlig omkostningseffektivt fototilbud • Manuel opretning af et barn i systemet, med manglende datavalidering
	Muligheder	Trusler
Eksterne	• Nye samarbejdspartnere • Flere jobs til fotografer	• Skadelige aktører • Diverse konkurrenter af skolefoto firmaer (i København) • Anmeldelser - forældre, der klager, og evt. forhindrer andre forældre i at købe og bestille fotos, eller de går til en konkurrent i stedet. • Virksomheden går konkurs, hvis Vicki løber tør for penge. • Problemer med samarbejdspartnere

³ (Det skal du vide om SWOT-analyse, n.d.)

Inception Deck (Projekt-"Pitch") (fælles)

Vi har brugt Inception Deck⁴ for at sørge for at alle involverede i projektet har en fælles forståelse for projektets vision.

1. *Why are we here?* (Hvad er det brændende problem?)

Vi er blevet kontaktet af Vicki fra Skolefotograferne, som ønsker et nyt og forbedret skolefoto-system, til både forældre, skoler, elever og fotografer.

Vi ønsker at skabe en fleksibel og brugervenlig skolefoto-oplevelse for skoler, fotografer og især forældre, når deres børn skal have taget skolefotos. Der er efterspørgsel på en personlig og skræddersyet skolefoto-oplevelse. Dette realiserer vi med vores system, som giver forældre mulighed for nemt selv at bestille skolefotos af deres barn, hvor de kan præcisere specifikke krav, så som om de ønsker at købe fotos digitalt eller fysisk, hvilken størrelse de skal være, hvor mange de ønsker, m.m.

2. Elevator pitch

Vi har brugt skabelonen i bilag 2 til at udarbejde følgende elevator pitch:

[FOR] Vicki fra Skolefotograferne [SOM] ønsker et nyt og forbedret skolefoto-system, [ER] *Smilfotograferne.dk* et fleksibelt og brugervenligt system, [DER] giver forældre, skoler og fotografer mulighed for nemt og gnidningsfrit at holde overblik over hele skolefoto-processen. [I MODSÆTNING TIL] det gamle system og vores konkurrenter, [ER VORES PRODUKT] brugervenligt og giver en moderne booking-oplevelse, som samtidig sikrer at forbindelsen mellem skole, fotografer og forældre fungerer problemfrit.

⁴ (The Agile Inception Deck, 2010)

3. The Product Box (Visualisering af produktæskén)

Forside:

Produktnavn: Smilfotograferne.dk

Tagline: "Vi fanger smilene, før de løber væk"

Nøglefunktioner: (Frederik, Kasper og Maria)

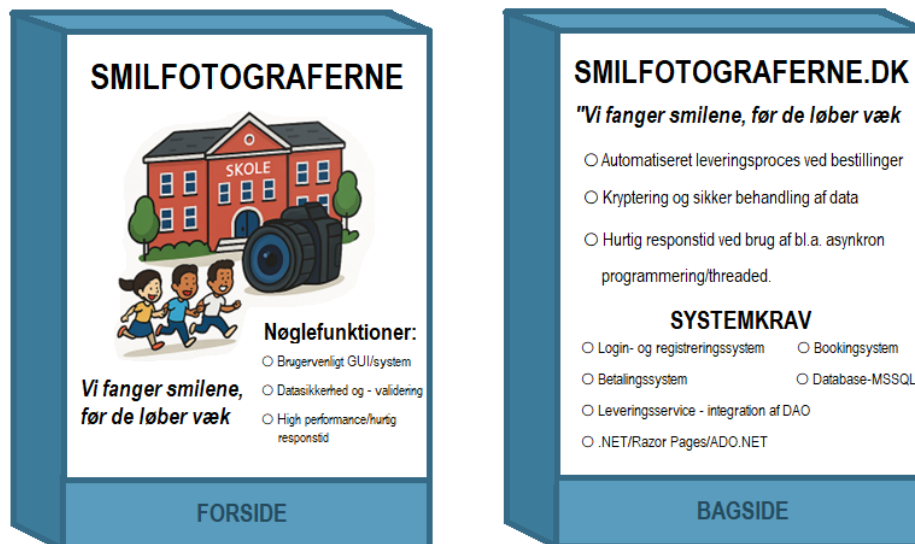
- Brugervenlig GUI/system
- Datasikkerhed og - validering
- High performance/hurtig responstid

Bagside: (Frederik)

- Automatiseret leveringsproces ved bestillinger
- Kryptering og sikker behandling af data
- Hurtig responstid ved brug af bl.a. asynkron programmering/threaded

Systemkrav: (Frederik)

- Login- og registreringssystem
- Bookingsystem
- Betalingssystem
- Leveringsservice - integration af DAO
- Database - MS SQL
- .NET/Razor Pages/ADO.NET



Udarbejdet delvist med AI og i Paint af Maria

4. The "Not-List" (Hvad laver vi ikke i dette projekt? Vigtigt for afgrænsning) (Frederik)

Out of scope

- Ingen supportchat/help desk
- Udelukkende skolefotos - ingen booking til events, begivenheder, happenings etc.
- Udenlandsk kundesegment, f.eks. kompatibilitet med udenlandske skoleloginsystemer, etc.
- Betalingssystem
- Leveringstjeneste
- (Private fotosessions)

5. *Meet the Neighbors* (Hvem er interessenterne? F.eks. bestyrelsen, menige medlemmer, kassereren) (Maria)

Ifølge casebeskrivelsen⁵ er der følgende interessenter i systemet med tilhørende opgaver:

Fotograf:

- Fotografens opgaver er at vedligeholde deres egen profil og uploade de færdige billeder (portrætter, gruppebilleder osv.) til systemet. Desuden skal de have adgang til detaljer om planlagte fotosessioner.

Forældre:

- Forældrenes opgaver er at oprette sig selv og deres børn i systemet. Desuden skal de kunne bestille og betale for billeder i systemet.

Skolelærer:

- Skolelærernes opgaver er at booke tidsintervaller til deres klasser, når datoerne for skolefoto er fastlagt, samt godkende og verificere elevernes navne. Desuden skal de kunne ændre bookingtider for klassen, hvis der opstår ændringer i skemaet.

Skolesekretær:

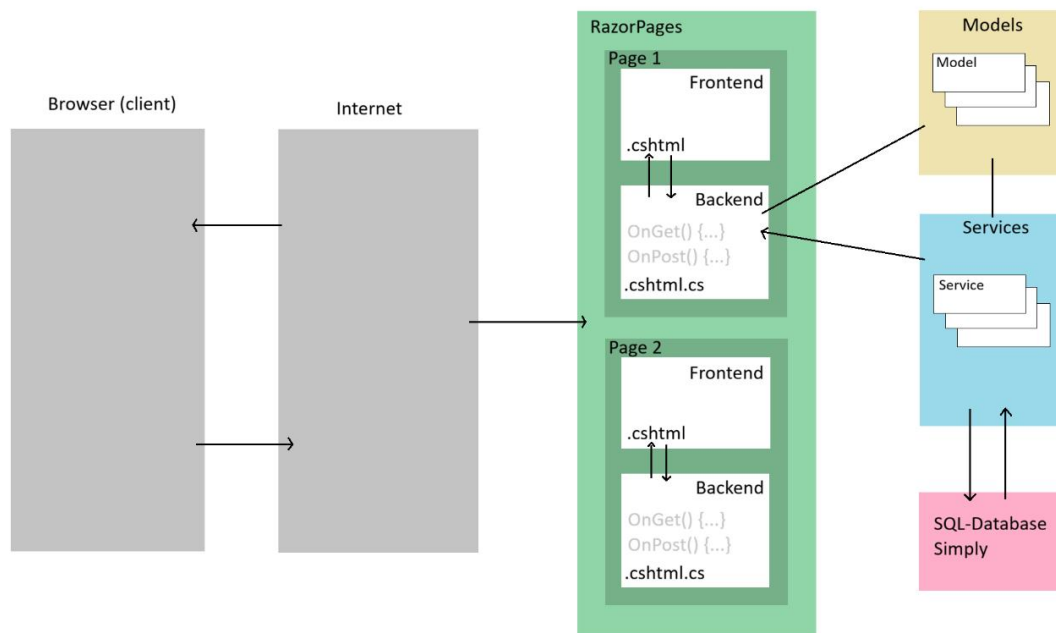
- Skolesekretærens primære opgave er at agere kontaktperson i forbindelse med oprettelse og styring af fotosessioner. Desuden er de ansvarlige for produktbestillinger, samt at koordinere tidsplaner med lærere, fotografer og andre interessenter.

⁵ (Skolefoto-servicesystem 2026, 2026)

Systemadministrator (Vicki):

- Systemadministrators opgaver er at administrere platformens indstillinger, brugerkonti og tekniske aspekter, samt oprette skoler og deres skolesekretærer i systemet. Desuden har systemadministrator overblik over alt backend og data, såsom bookinger, fotografer, elever m.m.

6. Show The Solution (En overordnet beskrivelse af det system, I vil bygge) (Maria)



Figur over vores system

Ovenstående model viser en overordnet beskrivelse af vores system, og hvordan brugerens browser taler sammen med serveren.

Det meste af vores system kører på serveren – dvs. alt fra vores internet, til vores RazorPages til vores Models, Services og Database.

Vores RazorPage har en foranliggende "View"-del, som vi har kaldt Frontend, og en bagvedliggende Backend. Backend består af to dele – en viewmodel, som stiller ting til rådighed til vores foranliggende side (fx public properties), og en controller som har

en OnGet og en OnPost metode, som hhv bruges til at vise noget og ligge data ned i vores database, dictionaries eller repositories.

Når vores applikation startes op, vises vores Frontend index-side ude i vores browser. Her sendes en forespørgsel til et link, fx CreateParent, som så gennemgår vores bagvedliggende sides constructorer, OnGet og OnPost metoder ovre på serveren. Herefter genereres vores Frontend view, hvor alt vores C# fjernes og laves om til html, og igen bliver sendt tilbage til vores browser. I browseren er der så fx en submit knap, som sender vores data fra formen tilbage til vores bagvedliggende Backend-side. På den måde kører vi hele tiden frem og tilbage mellem klient (browser) og server (internet), og alt vores C# afvikles på vores server.

7. What Keeps Us Up At Night (Risiko analyse over projekts udvikling) (Kasper, Sinan, Frederik og Maria)

Risiko	Sansynlighed 1-5	Konsekvens 1-5	S*K	Løsning
Et gruppemedlem bliver syg	3	2	6	Aftale at mødes med gruppen igennem Zoom/Meet, hvis man er rask nok.
Kommunikation- og samarbejds problemer	2	5	10	- Få snakket oftere i møder og kommunikerer generelt i processen - Få snakket tingene igennem, hvis der opstår tvivl, eller uenigheder.
Problemer med at få afleveret til tiden	2	5	10	- Forsøge at holde os til planen, så vi kan nå vores deadlines

				- Gå på kompromis med visse features, vi evt. ikke kan nå at færdigudvikle.
Manglende testing af det færdige produkt	3	4	12	- Lav unit tests - Udfør alle unit tests

Vi har lavet risikoanalyse⁶ for at identificere potentielle trusler, der kan opstå undervejs i vores projektudvikling. For hver trussel har vi beregnet risikofaktoren ved først at estimere sandsynlighed og konsekvens for den pågældende risiko på en skala fra 1-5 og efterfølgende ganget de to værdier. For at mitigere risikoen er vi til sidst kommet med et bud på en løsning.

8. Size it up (fælles)

Epic	Størrelse (Size It Up)	Forventet Tidsramme (uger)	Sprint
Virksomhedsanalyse	L	Uge 12 - 17	0
Systemudvikling	M	Uge 17-18	1-2
Backend	XL	Uge 17-22	1-4
Frontend	L	Uge 17-22	1-4
Unit Testing	S	Uge 21-22	3-4
Brugertest	XS	Uge 22	4

⁶ (Bering, 2020)

Vi har udarbejdet ovenstående diagram for at have et groft estimat over vores projekts størrelse og kompleksitet. I venstre side ses vores Epics, som vi her har delt groft op i alt fra indledende analyse til backend, frontend og til sidst brugertest. Disse har vi estimeret størrelse og kompleksitet på fra X-Small til X-Large og forventet tidsramme estimeret i hhv. uger og hvilke sprints, vi har tænkt os at arbejde med dem i.

9. Be clear on what's going to give (Kasper, Maria og Frederik)

Tid, kvalitet og budget (i vores tilfælde har vi ikke noget budget) er altid faste elementer, der ikke kan rykkes på.

Tiden er fast, fordi vi har en deadline, der skal overholdes.

Kvaliteten er fast, fordi man aldrig skal gå ned på kvalitet og aflevere noget, man kan stå inde for.

Budgettet er fast, fordi PO har et budget, vi skal overholde, og vi kan derfor ikke regne med pludselig at få flere ressourcer løbende.

Scope er fleksibelt, og det eneste vi kan ændre på, da vi altid kan fravælge funktioner undervejs, hvis vi ikke har tid, budget eller de ligger uden for scope. Det er derfor det første, vi vil ofre, hvis vi kommer bagud i tidsplanen.

Tid: [X] Låst ●————— fleksibel

Kvalitet: [] Låst ●————— fleksibel

Budget: [] Låst ●————— fleksibel

Scope: [] Låst —————● fleksibel

Trade-off sliders til vores projekt⁷

⁷ (The Agile Inception Deck, 2010), s. 81

10. Show what it's going to take

Hvad har vi brug for for at løse opgaven? (fælles)

- Internetforbindelse
- Computer
- Brugertest
- Poul/Vicki til at få løbende feedback under projektet.

Eksterne programmer (Frederik)

- draw.io til diverse diagrammer/modeller
- Figma til hjemmeside, frontend prototype
- .NET/ADO.NET projekt i Visual Studio til at skrive kode (C#)
- Trello til bl.a. use stories
- GitHub/Git til version control
- MS SQL til database
- Simply.com til hosting af database og hjemmeside

Analyse, design og implementering

Navneordsteknikken og tanker om domænet (Lukas, Frederik, Kasper og Maria)

Vi har benyttet navneordsteknikken til at finde de konceptuelle (fysiske og abstrakte) klasser, vores domænemodel skal indeholde. Klasserne repræsenterer de vigtigste begreber og objekter i domænet og bruges til at skabe overblik over, hvad systemet skal indeholde og håndtere.

Fysiske	Abstrakte
• Forælder • Barn • Fotograf • Skolesekretær • Skolelærer • Admin • Skole • Produkt	• Klassebooking • Familiebooking • Ordre • Login

Herefter fandt vi de attributter, som hver konceptuelle klasse skulle have:

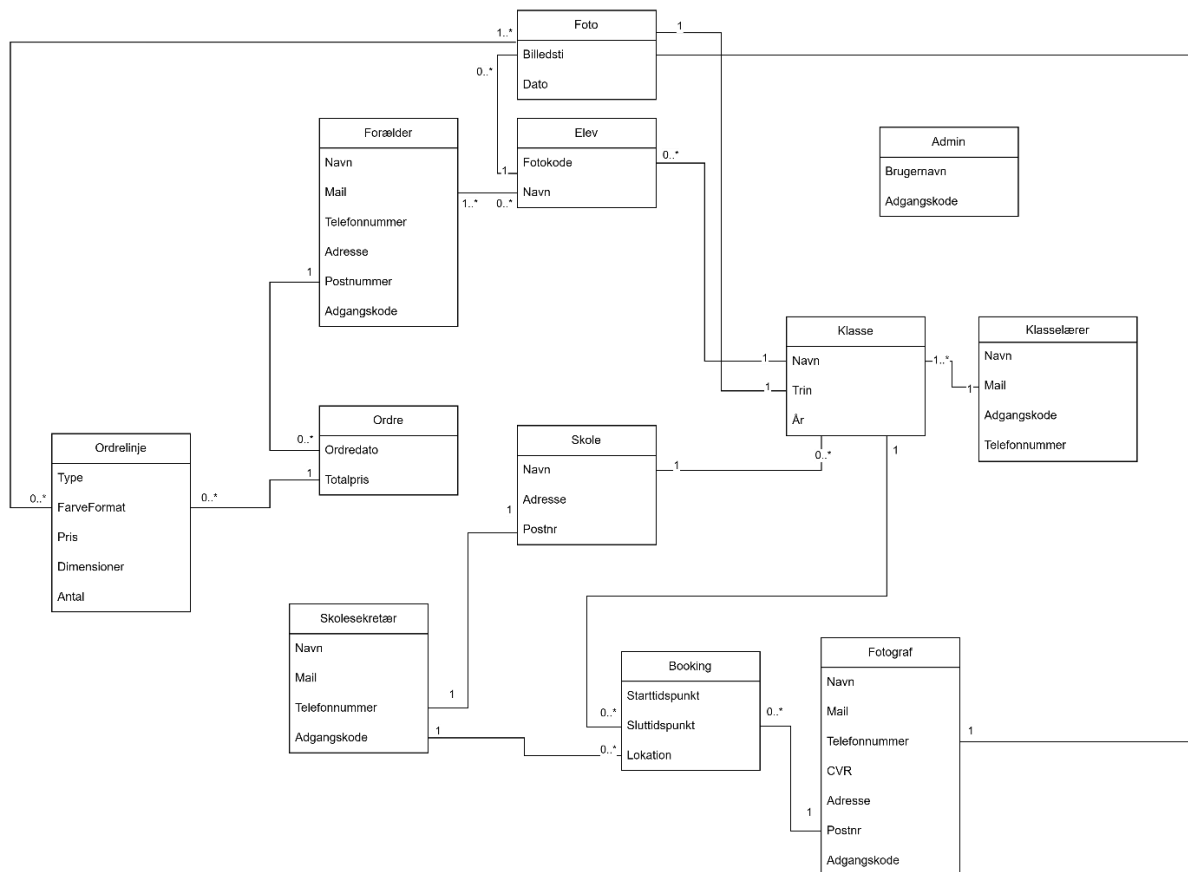
Forælder	Barn
• Navn • Mail • Telefonnummer • Adresse • Postnr • Adgangskode	• Id (bruges som nøgle af forældre) • Navn • Skole • Klassetrin • Årgang • Foto

Fotograf	Skolesekretær
• Navn • Mail • Telefonnummer • CVR • Adresse • Postnr • Adgangskode	• Navn • Email • Adgangskode • Telefon • Skole
Skolelærer	Admin
• Navn • Email • Adgangskode • Telefon • Klasse • Skole • Foto	• Brugernavn • Adgangskode
Klassebooking	Familiebooking
• Starttidspunkt • Sluttidspunkt • Klasse • Skolelærer • Fotograf • Lokale • Skole	• Starttidspunkt • Sluttidspunkt • Barn • Fotograf • Lokation
Ordre	Skole
• Totalpris	• Klasse(r)

• Fotoantal	• Årgang(e) • Adresse • PostNr • Skolesekretær • Lærer(e) • Antal elever
Produkt (evt. enum)	
• Filnavn • Type • Størrelse • Pris • Farve	

Domænemodel (Frederik, Lukas, Kasper og Maria)

Med udgangspunkt i ovenstående refleksioner omkring vores konceptuelle klasser, har vi udarbejdet følgende domænemodel:



Domænemodel over systemet

Overvejelser og tanker om domænemodel, konceptuelle klasser og relationer (Frederik, Lukas, Kasper og Maria):

Vi har en **Skole** med et navn, en adresse og et postnr. Vi har ikke givet skolen et telefonnummer og en mail, fordi vi tænker, at al kontakt foregår gennem skolesekretæren. **Skolen** har en relation til **Klasse**, og kan have 0 til mange klasser. Skolen har også en relation til **Skolesekretær**, og kan have 1 skolesekretær. Skolen har en relation til **Klasselærer**, og kan have 0 til mange klasselærer.

Vi har en **Klasse** med et navn (a, b, c), trin og årgang. Klassen har relation til **Skole**, og kan være tilknyttet en skole. Klassen har relation til **Klasselærer**, og kan være tilknyttet én klasselærer. Klassen har relation til **Elev**, og kan have 0 til mange elever. Klassen har relation til **Booking**, og kan have 0 til mange bookinger (gennem årene). Klassen har relation til **Foto** (klassefoto), og kan have et foto tilknyttet.

Vi har en **Klasselærer** med et navn, mail, adgangskode og telefonnummer.

Adgangskoden bruges, når klasselæreren får oprettet sit brugerlogin. Brugernavnet bliver automatisk mailadressen (det gør den for alle brugere på nær Admin).

Klasselæreren har relation til **Klasse**, og kan have 1 til mange klasser, da vi antager at de kan være klasselærer i flere klasser. Klasselæreren har relation til **Foto**, og kan have et foto tilknyttet (medarbejderportræt).

Vi har en **Skolesekretær** med et navn, mail, telefonnummer og adgangskode.

Skolesekretæren har relation til **Booking**, og kan have 0 til mange bookinger.

Vi har en **Fotografering** med et starttidspunkt, sluttidspunkt og en lokation.

Fotografering har relation til **Skolesekretær**, og kan have en skolesekretær tilknyttet.

Fotografering har relation til **Fotograf**, og kan have en fotograf tilknyttet. Fotografering har relation til **Klasse**, og kan have en klasse tilknyttet.

Vi har en **Fotograf** med et navn, mail, telefonnummer, CVR, adresse, postnr og adgangskode. Fotograf har relation til **Booking**, og kan have 0 til mange bookinger.

Fotograf har relation til **Foto**, og kan have 0 til mange fotos.

Vi har en **Elev** med en fotokode og et navn. Vi har valgt ikke at give eleven flere attributter, fordi vi tænker, at de er overflødige for systemet (fx fødselsdato). Vi har valgt at give eleven en fotokode, som de får udleveret ved fotograferingen. Fotokoden gives til forældrene og bruges, når de skal bestille billederne. Eleven har relation til **Klasse** og kan være tilknyttet én klasse. Eleven har relation til **Foto** og kan være tilknyttet 0 til mange fotos. Eleven har relation til forælder og kan være tilknyttet 1 til mange forældre (da der findes mange forskellige familiekonstellationer i dag).

Vi har en **Forælder** med et navn, mail, telefonnummer, adresse, postnr og adgangskode.

Forælder har relation til **Elev** og kan være tilknyttet 0 til mange elever. Forælder har relation til **Ordre** og kan være tilknyttet 0 til mange ordrer.

Vi har et **Foto** med en billedsti og en dato. Foto har relation til **Elev** og kan have én elev

tilknyttet. Foto har relation til **Fotograf** og kan have én fotograf tilknyttet. Foto har relation til **Ordrelinje** og kan have 0 til mange ordrelinjer.

Vi har en **Ordre** med en ordredato og en total pris. Ordre har relation til **Forælder** og kan have én forælder tilknyttet. Hvis flere forældre skal bestille det samme billede, gør de det på flere ordrer. Ordre har relation til **Ordrelinje** og kan være tilknyttet én til mange ordrer.

Vi har en **Ordrelinje** med en digital (sandt/falsk om fotoet skal være digitalt eller ej), størrelse, pris, sort/hvid (sandt/falsk om fotoet skal være sort/hvid eller farvet) og antal. Ordrelinje har en relation til **Ordre** og kan være tilknyttet én ordre. Ordrelinje har en forbindelse til **Foto** og kan være tilknyttet ét foto.

Vi har en **Admin** med et brugernavn og en adgangskode, som har en generel relation til alle konceptuelle klasser, men vi har valgt ikke at vise den i vores domænemodel.

ER-diagram (Lukas, Frederik, Kasper og Maria)

Vi har lavet vores ER diagram og fulgt 1NF ved at undgå "atomiske" værdier⁸.

Vi opbevarer en ParentStudent relation i sin egen tabel for at forblive i 1NF.

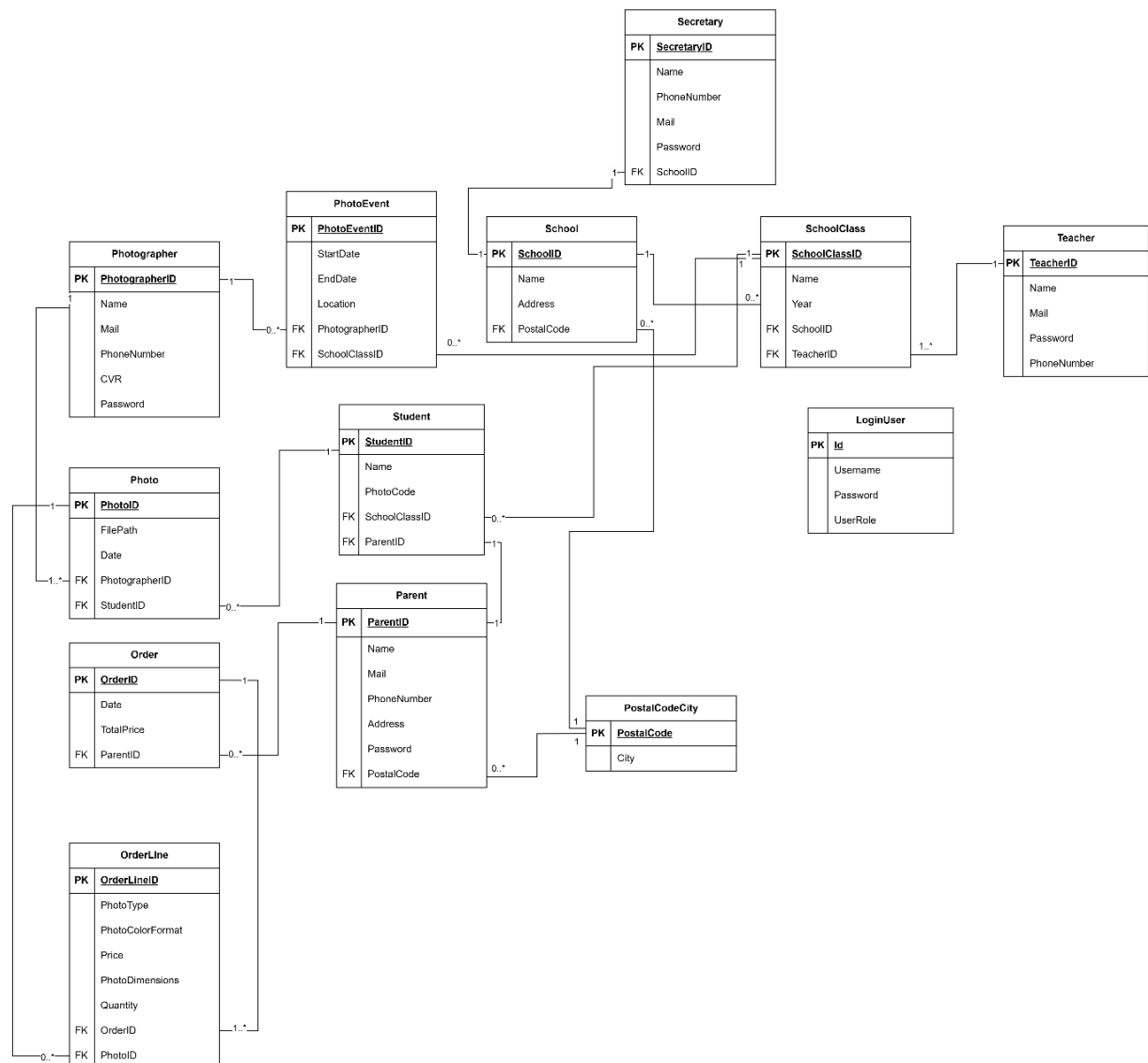
Vi har lavet en OrderLine, som fungerer som en mellemtabel, fordi en ordre kan indeholde flere fotos, og det samme foto kan optræde på mange forskellige ordrer. Der er derfor tale om en mange-til-mange relation, som kun kan håndteres med en mellemtabel (ordrelinje).

Samme gør sig gældende for ParentStudent tabellen, fordi en forælder kan have mange børn, og et barn kan have mange forældre - igen en mange-til-mange relation, som håndteres og løses med en mellemtabel (ParentStudent).

⁸ (Heegaard, 2026)

For at overholde 3. normalform, har vi lavet en separat tabel til PostalCode og City.

Dette fordi man altid kan udlede postnummer ud fra by og omvendt.

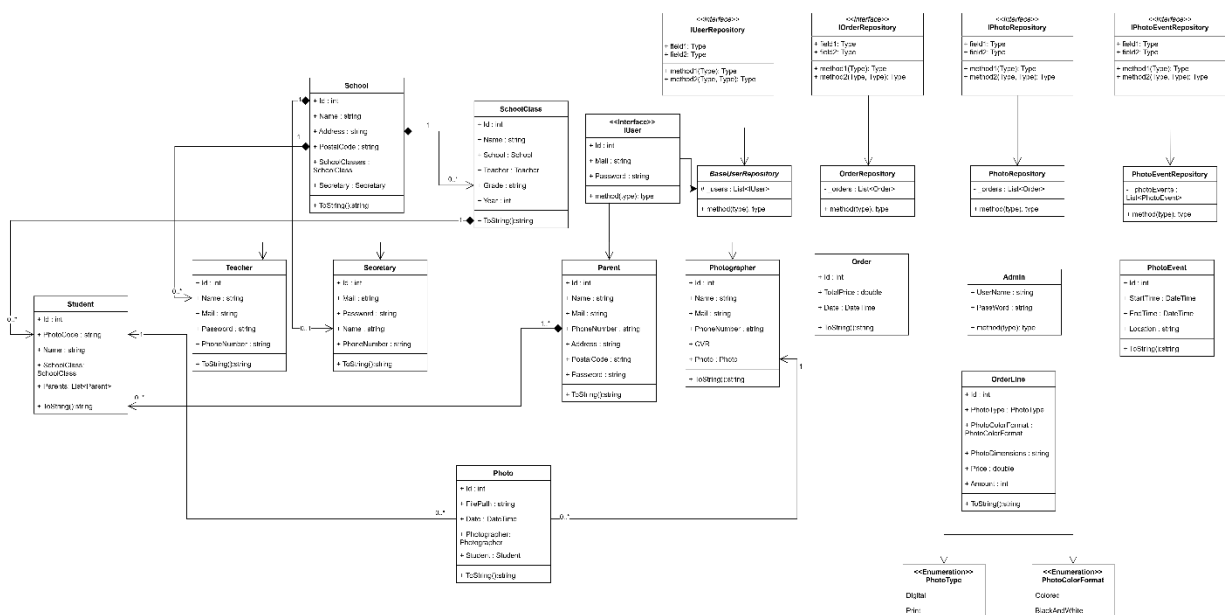


ER-diagram over systemet

Klassediagram (diskuteret og udarbejdet fælles)

Første udkast (skrevet af Lukas)

Link til [Draw.io](#) med vores klassediagrammer (der er lavet flere tabs til forskellige versioner).



Første udkast af klassediagram

Her ses første udkast af vores klassediagram.

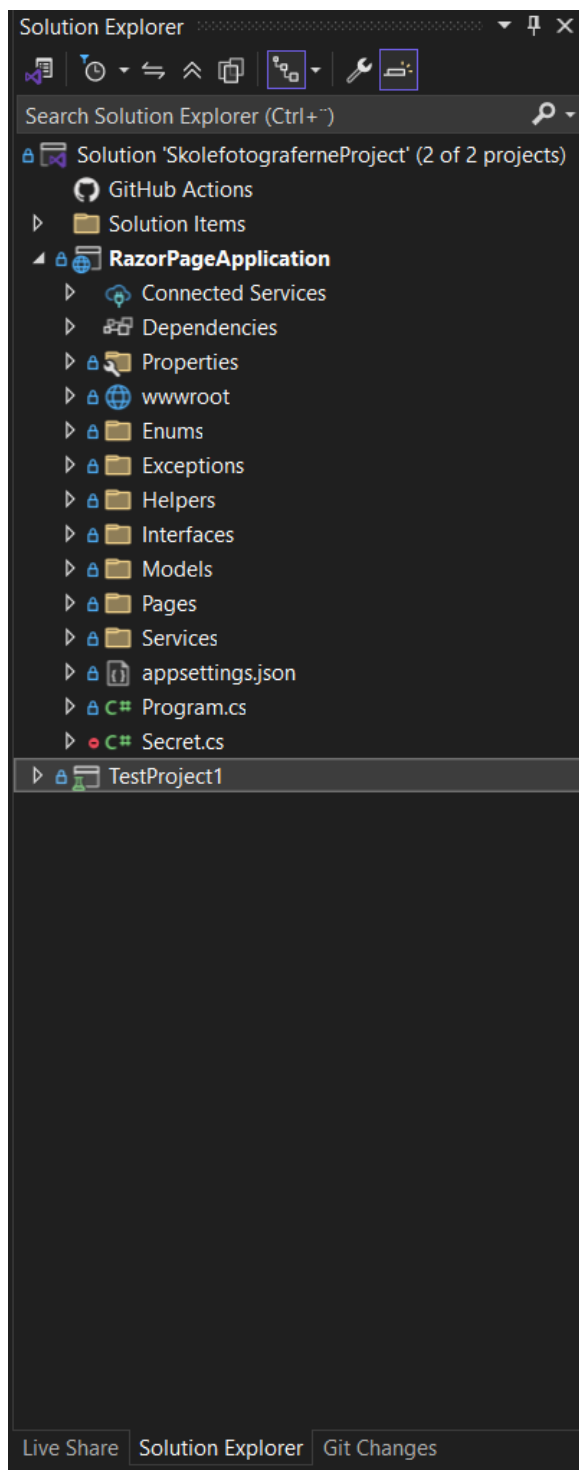
Vi har valgt at gøre get/set implicit. Der er nogle kompositioner. Fx mellem School og Teacher, hvor School er en helhed og Teacher er en del. Slettes helheden, så slettes delen også, der er altså tale om et forhold, hvor den ene ikke eksisterer uden den anden.

Der er også en pil mod Teacher. Dette betyder at skolen har 0 til mange Teachers som attribut altså i designet har skolen en liste af teachers.

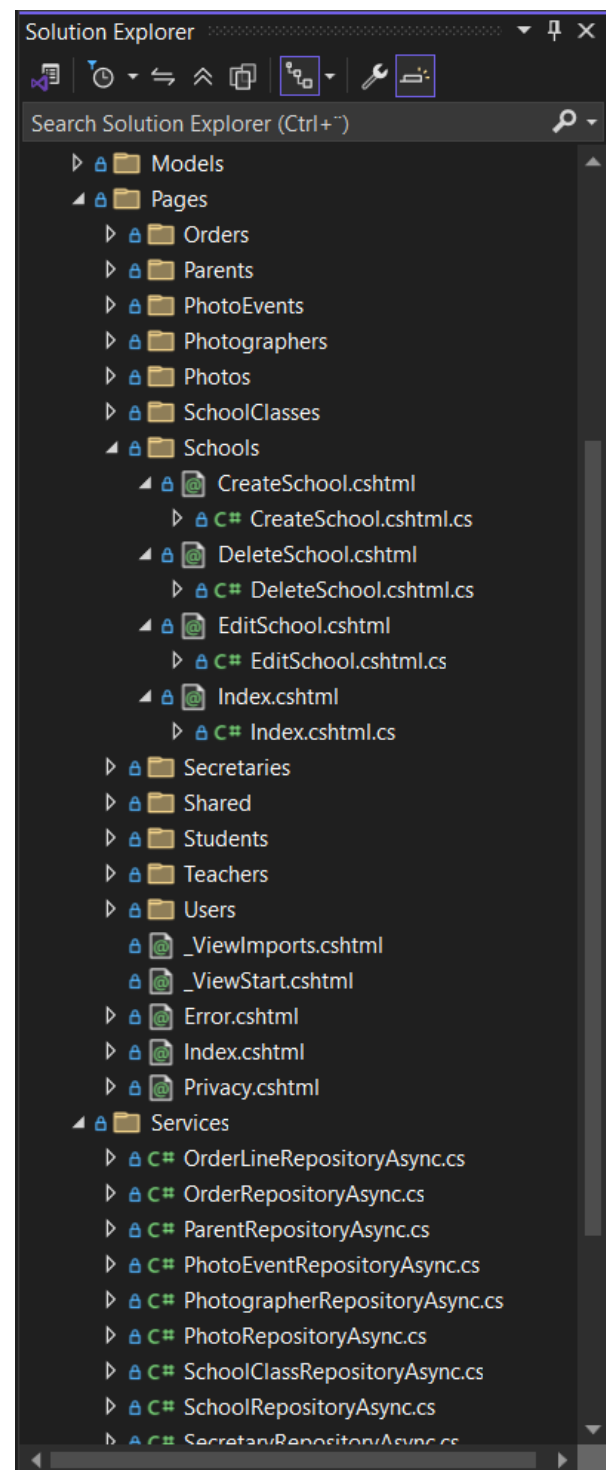
Derudover har vi nogle interfaces til hvert repository som implementerer disse.

De fleste modelklasser følger ikke noget interface, men alle former for brugere gør. Fx Teacher implementerer IUser. Bemærk at student ikke gør, da elever ikke får deres eget login, men blot er en enhed for hver forælder.

Projektopbygning – mappestruktur i Solution Explorer (Frederik)



Screenshot af vores Solution Explorer



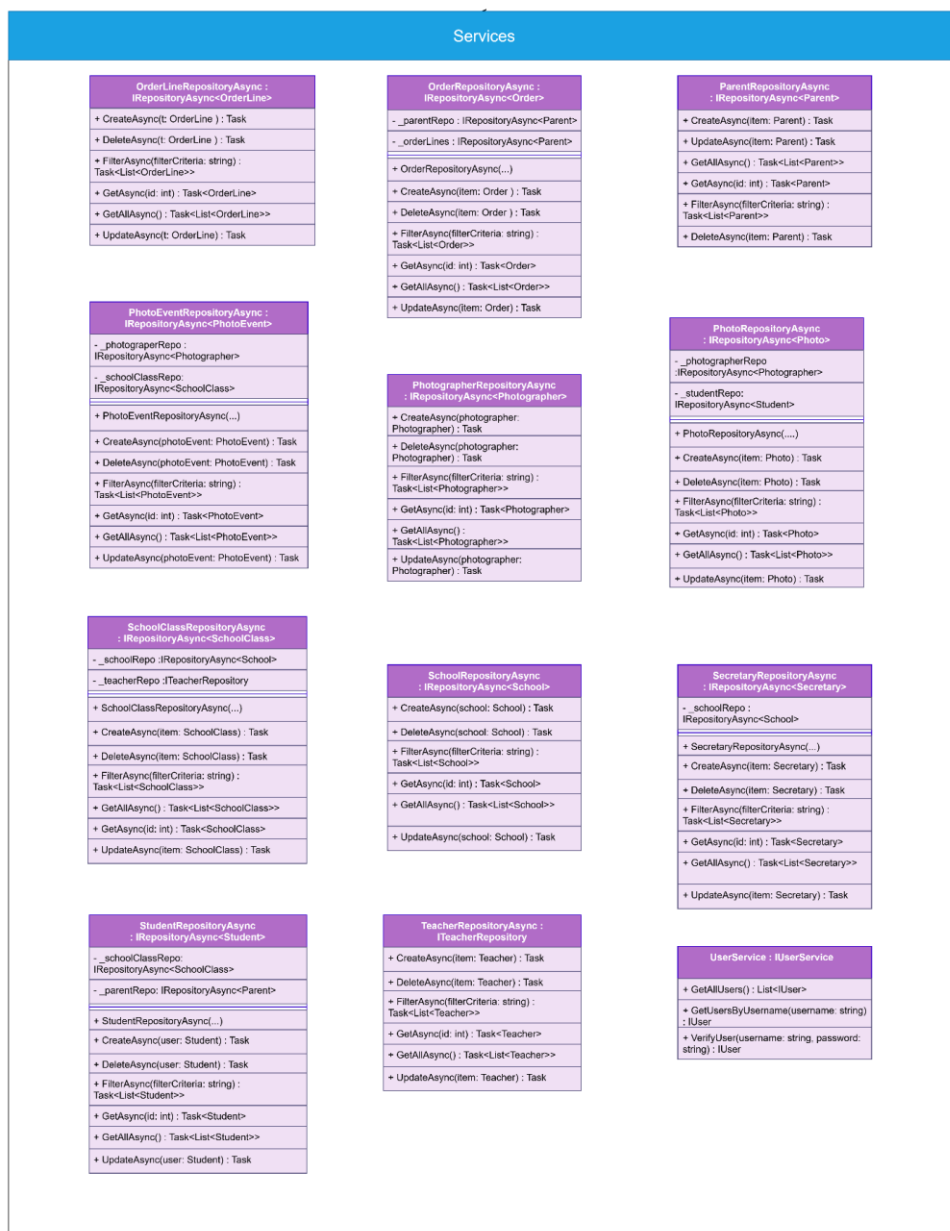
Screenshot af vores Solution Explorer - med åbne mapper

Hele vores projekts kode er lagt under et samlet .csproj. Dette blev vi anbefalet, da det bl.a. simplificerer unit testing.

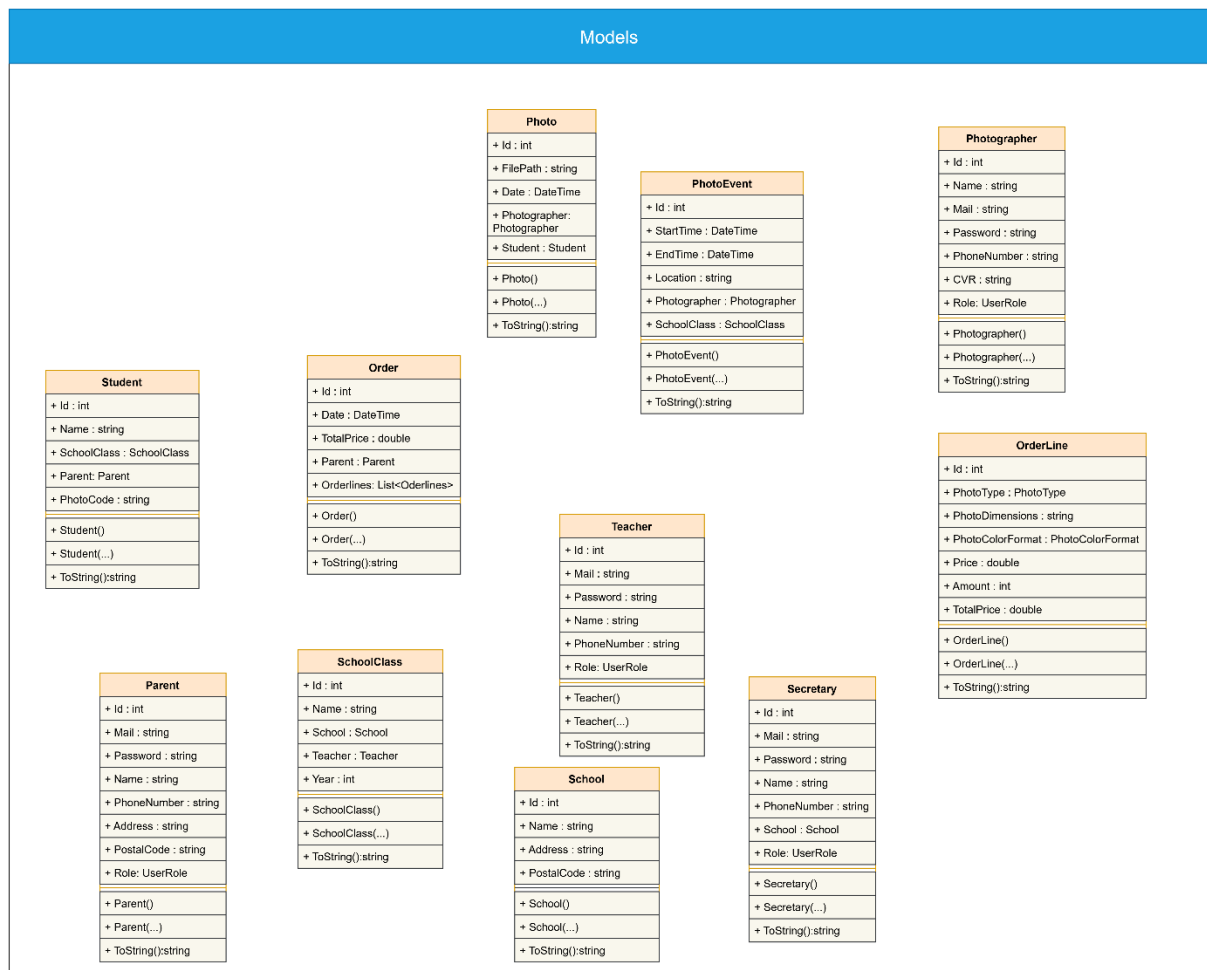
Vores klasser er opdelt i mapper ud fra deres formål, og om de tilhører 'Library' delen eller RazorPage delen af projektet.

Det endelige klassediagram – uden relationer (Kasper, Sinan og Maria)

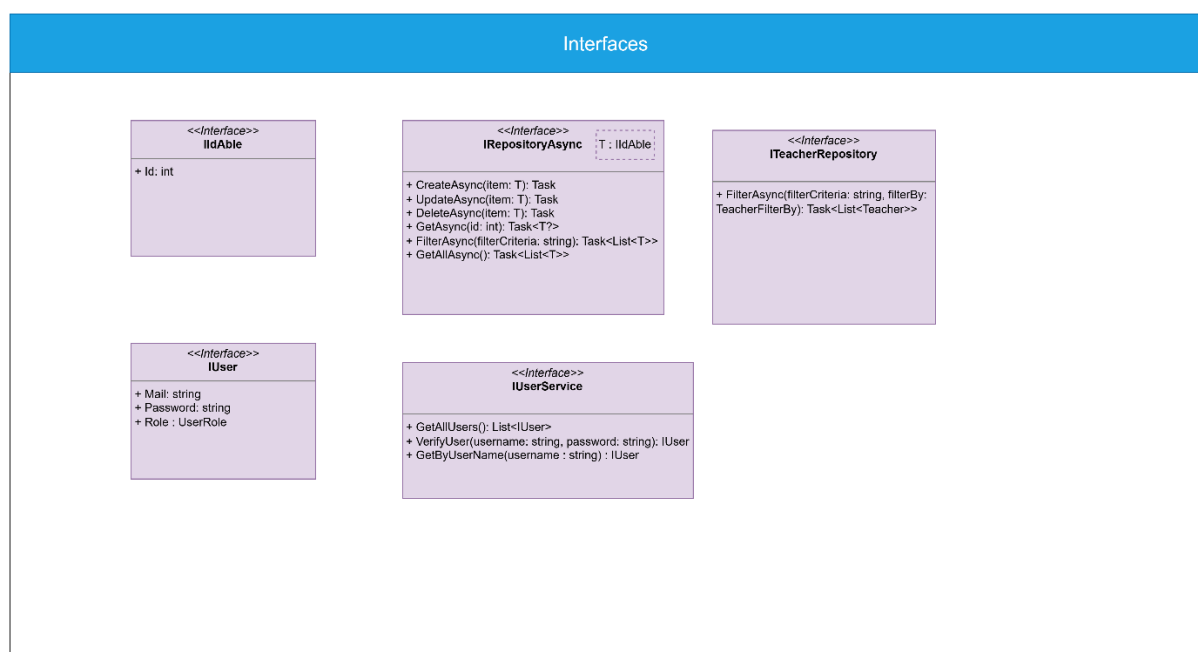
Vi har brugt Draw.io til at lave et klassediagram over vores system. Dette for at skabe et visuelt overblik over systemets struktur og de funktionaliteter, det skal have. Nedenfor ses hver enkelt mappe og dets klasser med instance fields, properties og metoder. I næste afsnit ses klassediagrammet med tilhørende relationer.



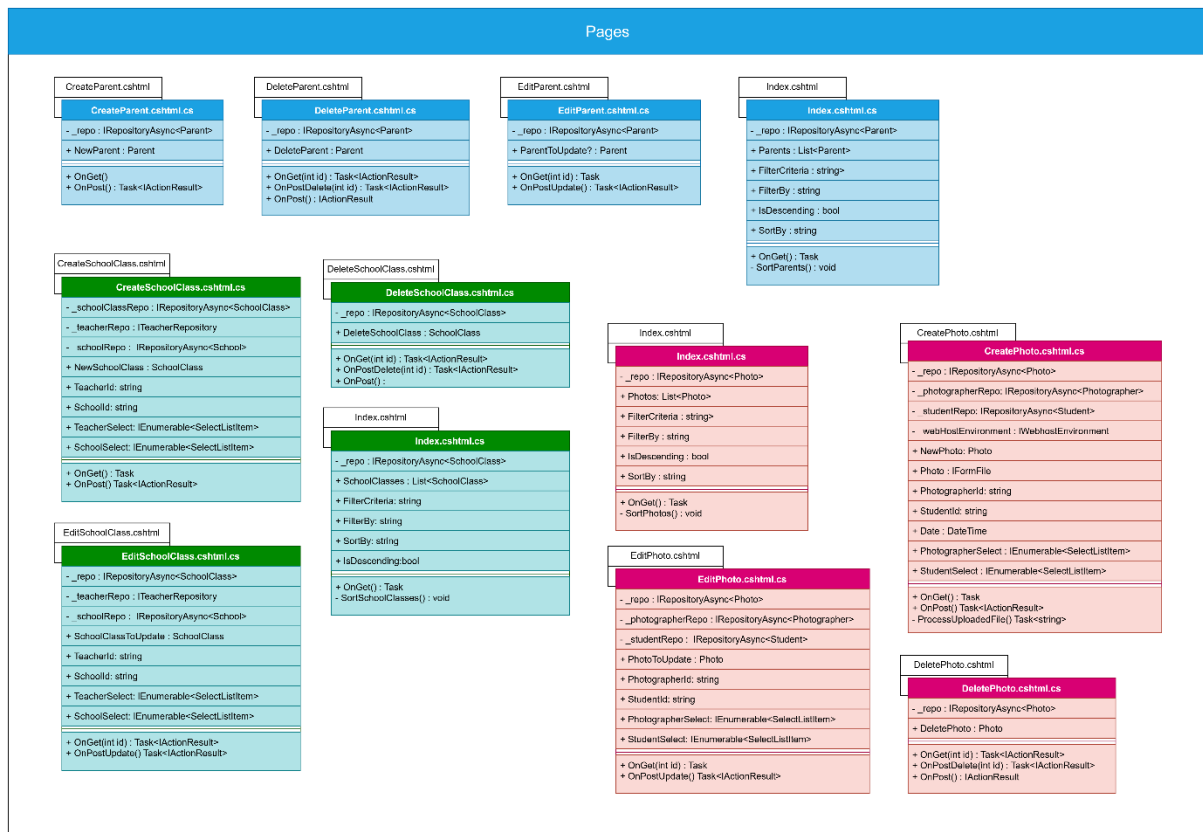
Her ses vores mappe Services, som indeholder alle vores repositories.



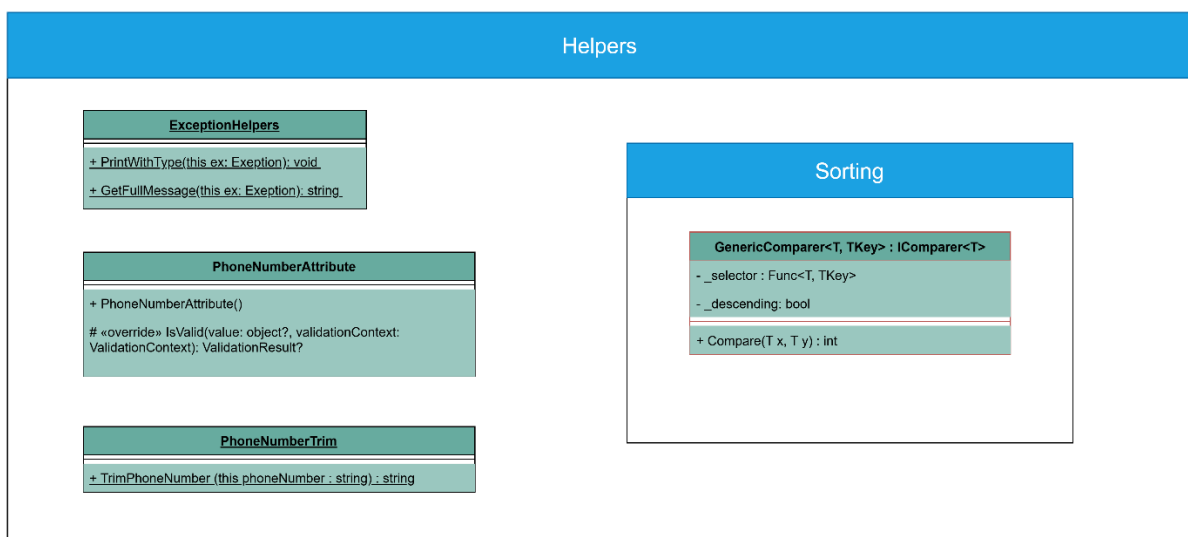
Her ses vores mappe *Models*, som indeholder alle vores modelklasser.



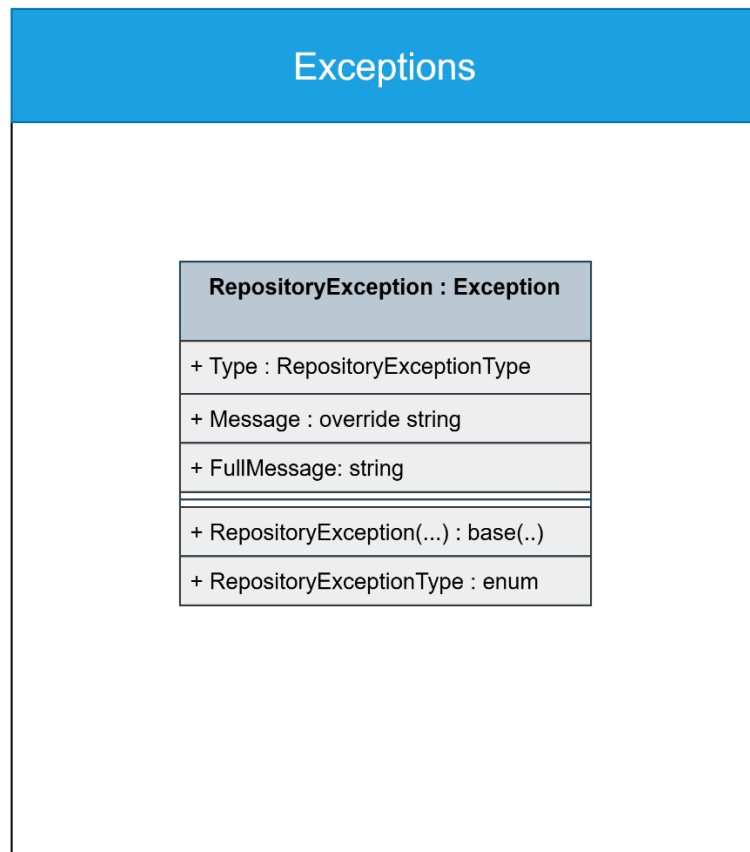
Her ses vores mappe *Interfaces*, som indeholder alle vores interfaces.



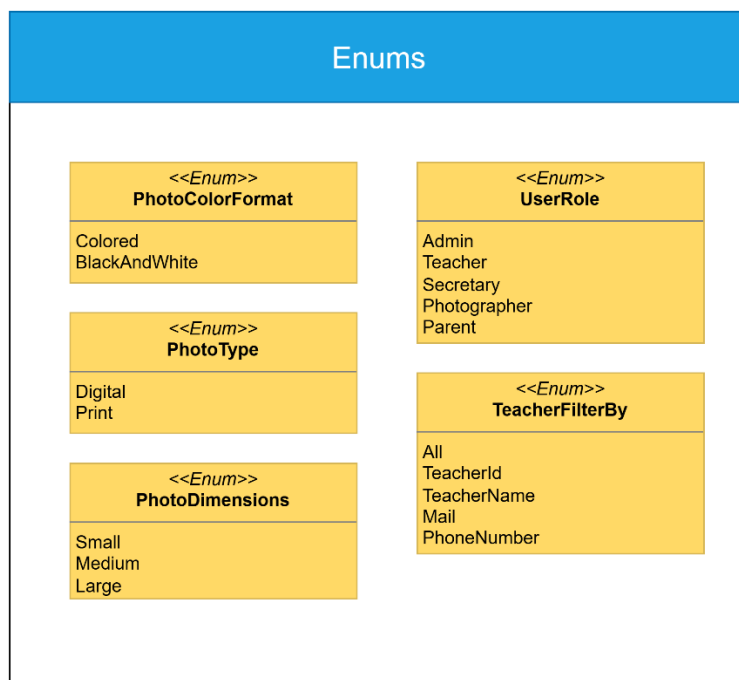
Her ses vores mappe *Pages*, som indeholder alle vores RazorPages. Vi har illustreret den bagvedliggende .cshtml samt den foranliggende .cshtml.cs med dens properties og metoder.



Her ses vores mappe *Helpers*, som yderligere indeholder en *Sorting* mappe, og diverse helper klasser.



Her ses vores mappe *Exceptions*, som indeholder vores exception-klasse.



Her ses vores mappe *Enums*, med alle vores enumerations.

Klassediagram over systemet1



Lukas:

I klassediagrammet har vi opdelt de forskellige klasser efter kategori. Pagemodels er en kategori i sig selv. Disse får fat i deres repository via dependency injection, så disse kan tilgås eksternt og stilles til rådighed af program.cs i builder.

Selve frontendsiderne er ikke med i klassediagrammet. Blot deres modelsider.

Sinan:

Vores service klasser implementerer de fleste repositories det samme interface som har de CRUD metoder der skal bruges, hvilket gøre det nemmere og hurtigere at implementer dem, end hvis man skulle gøre det manuelt.

Under mappen Exception har vi vores skræddersyede Exception klasse, som arver fra klassen Exception. Den er der til, hvis en fejl dukker op alt efter hvilken Enum Type der får fejlen som oprettelse af en lærer eller redigering af en sekretær.

Enum mappen indeholder forskellige enum klasser der har unikke værdier til f.eks hvis man er logget ind som en Admin vil du kunne se ting som kun admin kan, eller til filtrering af Teacher. Mappen er forbundet til vores Model klasser, da det er vores modelklasser, som benytter vores enums.

Under mappen Models ses vores modelklasser, som indeholder klassernes struktur som ID, Navn, Password osv for sekretær. Mappen er forbundet med Services mappen og Interfaces mappen, da Services klasser bruger modelklasserne til at opbevare de korrekte værdier, og interfaces fordi nogle bliver benyttet på hver modelklasse som IUser og IIdAble.

Til sidst ses vores mappe Pages, som indeholder et udsnit af vores RazorPages, hvor man kan se den foranliggende .cshtml og den bagvedliggende .cshtml.cs med dens properties og metoder.

Userstories (udarbejdet fælles, skrevet af Maria)

Her er et udpluk af vores userstories fra vores [Google Sheets](#). Vi har desuden brugt [Trello](#) til at arbejde med vores userstories, fordi det er et godt værktøj til at skabe overblik og efterhånden flytte userstories fra vores Sprint Backlog over i Done.

Vores arbejde med userstories vil blive uddybet yderligere i vores afsnit ”Udviklingsproces”.

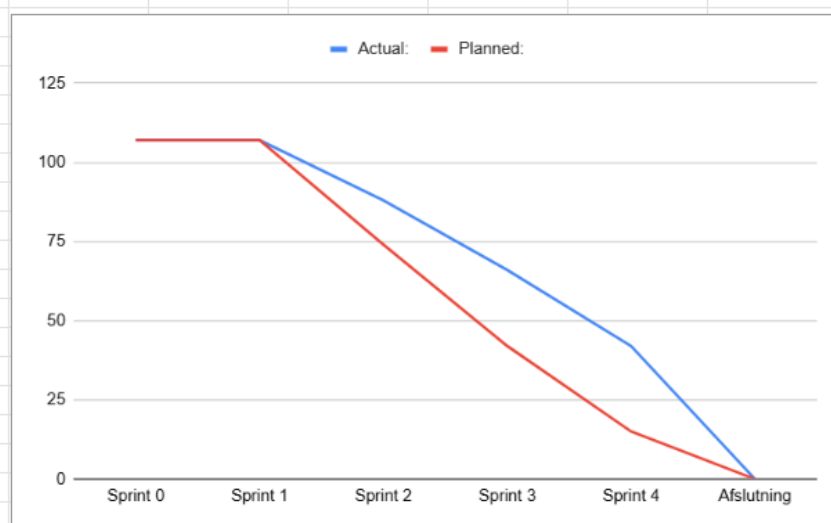
▼	Tr Userstory	▼	Tr Acceptance criteria	▼	Tr Tasks	▼	# Storypoints	▼
	[Admin] login						1 - 5	
1	Som admin, ønsker jeg et admin-login, så jeg kan tilgå adminpanelet.		- Skal tilgås som en URL, så den almene bruger ikke kan finde det. - Admin skal kunne logge ind og få adgang til adminpanelet		- Opret side til loginpanel - Implementer loginfunktionalitet - Opret login til admin		2	
	[Admin] Forældre:							
2	Som admin, ønsker jeg at kunne oprette forældre, så jeg kan tilføje forældre til systemet.		- Admin skal kunne oprette forældre i systemet med navn, telefonnummer, email, adresse - Forældrene skal få auto-genereret et ID - Forældrene skal gemmes korrekt i database når de oprettes. - Systemet skal vise en fejlmeddelelse, hvis felterne ikke er udfyldt.		- Lav en Razorpage - Opret formular til oprettelse af forældre - Lav add metode i forældrerepository		1	
3	Som admin, ønsker jeg at kunne se en liste over forældre, så jeg kan danne mig et overblik over forældre.		- Admin skal få vist en liste over alle tilgængelige forældre i systemet		- Lav Razor Page til oversigt - Implementer visning af liste af forældre, ud fra opbevaret data - Lav GetAll i forældrerepository		1	
4	Som admin, ønsker jeg at kunne redigere forældre i systemet, så jeg kan holde forældres oplysninger up to date.		- Admin skal kunne redigere alle oplysninger for forældre - Oplysningerne skal opdateres og gemmes korrekt i databasen - Forælder-ID skal ikke kunne redigeres		- Lav en Razorpage, der indlæser data for den valgte forælder - Lav formular hvor man kan se og ændre på oplysninger - Lav update metode i forældrerepository		1	
5	Som admin, ønsker jeg at kunne slette forældre i systemet, så jeg kan fjerne forældre, der er oprettet ved en fejl, eller ikke længere er aktuelle.		- Admin skal kunne slette forældre fra systemet. - Forældren skal fjernes fra databasen. - Admin skal få vist en 'Er du sikker på, at du vil slette forældren?' advarsel, når vedkommende klikker på 'Slet'.		- Implementer sletning af data i databasen i repositoret - Implementer advarsels-popup i Razor Pages/front end		1	
6	Som admin, ønsker jeg at kunne filtrere forældre i systemet, så jeg kan finde forældre, der matcher et kriterie.		- Admin skal kunne filtrere forældre i systemet med X kriterier - Systemet skal filtrere forældrene korrekt efter kriteriet (fx fornavn)		- Implementer filtreringsfunktion i repositoret, som kan filtrere ud fra data i en database		2	
7	Som admin, ønsker jeg at kunne sortere forældre fra en specifik kolonne, så jeg kan		- Admin skal have en knap fx med en dropdown til at sortere forældrene fx		- Implementer alfabetisk sorteringsfunktionalitet i koden, f.eks.		2	

Udpluk af vores userstories fra Google Sheets

Burndown Chart (Kasper)

Vi har arbejdet med at lave et Burndown Chart (Rasmusson, 2010)⁹, på baggrund af vores estimering af vores userstories. Den måde vi har lavet vores Burndown Chart på, er ved at tælle alle vores point sammen, og derefter lave et estimat af, hvor mange userstories vi kunne nå at lave efter hvert sprint, svarende til en uge. Denne estimering er vist ved den røde graf i vores Burndown Chart. Vi har derudover også holdt regnskab over, hvor mange userstories vi har nået efter hvert sprint under vores arbejdsproces, det har vi gjort ved at regne alle de userstories vi nåede sammen, og sætte dem ind i vores excel-ark.

Actual		Planned		Planned beregning/fordeling af story points	
Sprint:	Story Point:	Sprint:	Story Points:	Sprint:	Points:
Sprint 0	107	Sprint 0	107	Sprint 0	0
Sprint 1	107	Sprint 1(Total effort)	107	Sprint 1	33
Sprint 2	88	Sprint 2	74	Sprint 2	32
Sprint 3	66	Sprint 3	42	Sprint 3	27
Sprint 4	42	Sprint 4	15	Sprint 4	15
Afslutning	0	Afslutning:	0	Total efforts:	107



Visning over beregning af Burndown Chart samt Burndown Chart over hele arbejdsprocessen

⁹ (Rasmusson, 2010), s.148-150

Estimering af userstories (Frederik, Lukas, Kasper og Maria)

Vi har valgt at bruge triangulation¹⁰ til at estimere vores userstories. Det har vi gjort, da det er en hurtigt og effektiv måde, hvor man starter med at vælge den største/sværeste userstory og sammenligner de resterende userstories med den. Man giver storypoints fra 1-5, hvor 5 angiver den mest komplekse userstory.

Vi estimerede, at userstorien "Som forælder ønsker jeg at kunne bestille billeder taget af mit barn og specificere varen jeg ønsker at modtage" er den mest komplekse userstory, da der hører mange tasks til den, der er mange forbindelser som skal laves med validering m.m.

Vi estimerede, at userstorien "Som forælder, ønsker jeg at kunne tilmelde mit barn i systemet, så jeg kan tilgå og evt. bestille fotos taget af mit barn." er den næstsværeste userstory, da der ligeledes hører mange tasks og forbindelser til den, og vi skal implementere en fotokode.

Vores CRUD (create, read, update, delete)- userstories vurderede vi til 1, da de er ret simple og hurtige at lave og ikke kræver nogle særlige forbindelser. Desuden kan vi efterhånden bare copy-paste dem, da metoderne og funktionaliteten går igen.

¹⁰ (Rasmusson, 2010)

	A	B	C	D	E
		Table1			
		Tr Userstory	Tr Acceptance criteria	Tr Tasks	# Storypoints
1		en meningstid rækker efter hvor jeg leder efter.		ascending, descending mm.	
54		Forældre			
55	44	Som forælder, ønsker jeg at kunne tilmelde mit barn i systemet, så jeg kan tilgå og evt. bestille fotos taget af mit barn.	- Barnet skal kunne tilmeldes i systemet - Når forælderen indtaster den udleverede fotokode, tilknyttes barnet automatisk forælderen	- Implementer funktion til at tilmelde barn til systemet - Implementer funktion til at "koble" forældre og elever sammen. - RazorPage side med formular til tilmelding af barn	4
56	45	Som forælder ønsker jeg at kunne oprette en bruger på siden, så jeg kan benytte systemet	- Forælderen skal kunne oprette en bruger, og angive mailadresse og password	- Implementer RazorPage - Implementer forælder brugeroprettelse i systemet	1
57	46	Som forælder ønsker jeg at kunne logge ind på siden når jeg har oprettet en bruger	- Forælderen skal kunne logge ind med deres email adresse og det password de har angivet.	- Implementer login RazorPage - Implementer loginfunktionalitet i systemet	2
58	47	Som forælder ønsker jeg at kunne bestille billeder taget af mit barn og specificere varen jeg ønsker at modtage	- Det skal være muligt at kunne lave en bestilling i systemet - Det skal være muligt at specificere hvilken slags billede man ønsker at modtage (sort/hvid eller farvet, størrelse, portræt og/eller klassefoto) - Forælderen skal modtage en bekræftelse på, at bestillingen er gået igennem	- Implementer ordreside i RazorPage - Implementer ordrefunktionalitet i systemet - Implementer mulighed for at ændre specifikationer for billedet, f.eks. sort/hvid, størrelse, antal, format mm. - Implementer pop-up boks i GUI med beskeden "Ordren er gået igennem (evt. email-bekræftelse)	5

Udsnit af vores Google Sheets dokument som viser vores estimerede Storypoints

GUI – prototype og tanker om front-end (udarbejdet i fællesskab, beskrevet af Frederik, Sinan og Maria)

Vores GUI og frontendens funktionalitet vil blive gennemgået yderligere

ID	Navn	Fotokode	Skoleklasse	Forælder	Slet Redigér
1	Emil Jensen	FOTO001	3A	Lars Jensen	Slet elev Redigér elev
2	Maja Christensen	FOTO002	5B	Pia Christensen	Slet elev Redigér elev

Opret ny elev

Navn
Indtast elevens navn her

Fotokode
Indtast elevens fotokode her

Skoleklasse
Vælg Skoleklasse

Forælder
Vælg Forælder

Opret en elev

[Gå tilbage til listen af elever](#)

Redigér elev

ID
1

Navn
Emil Jensen

Fotokode
FOTO001

Skoleklasse
1 - 3A - 2024

Forælder
1 - Lars Jensen - lars@gmail.com

Opdater elev

[Tilbage til elevoversigt](#)

Slet elev

ID: 1
Navn: Emil Jensen
Skole: FOTO001
Lærer: 3A
Forælder: Lars Jensen

Slet elev Annuler

Vi har brugt Jakob Nielsens 10 heuristikker til design af hjemmesiden og user-interface¹¹.

¹¹ (Nielsen, 2024)

Vi har brugt 1. heuristik, Visibility of system status, ved at give brugeren hurtig feedback efter at have trykket på en knap. Vi 'redirecter' brugeren, primært til foranliggende indeksside, når de f.eks. har oprettet en ny bruger korrekt.

Vi har fulgt 2. heuristik ved ikke at bruge internt programmeringssprog i vores frontend, men brugt hverdagsord, som alle kan forstå.

Vi har brugt den 3. heuristik, User control and freedom, ved at lave en sub-page til når man vil slette noget. Vi havde planlagt at lave en advarselsbesked og en pop-up, når man vil slette en sekretær eller lærer, men det havde vi ikke tid til.

Vi fulgte 4. heuristik, og var konsistente i navngivning af knapper, liste af brugerinformationer mm. Vi har også placeret sideelementer såsom knapper, tabeller mm. ens på alle undersiderne.

Vi fulgte 5. heuristik i form af error prevention, ved bl.a. at indføre dropdowns når man vælger f.eks. at oprette en fotograf, i stedet for manuelt at indtaste deres id. Ud over det har vi lavet validering af tekstlængde på f.eks postnummer og telefonnummer.

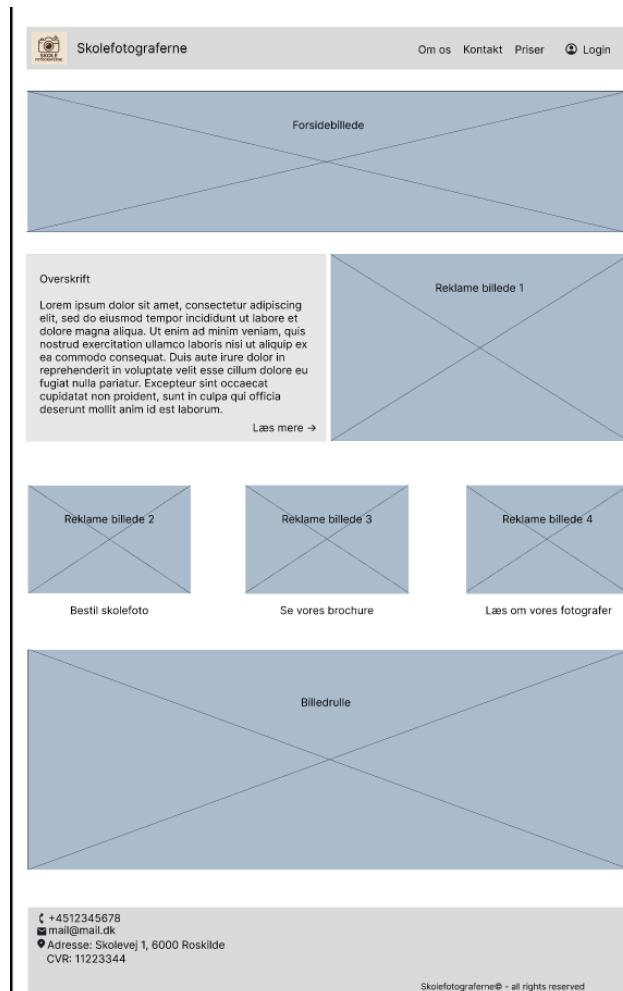
Vi fulgte 6. heuristik ved at lave 'placeholders' til inputfelter i oprettelse af f.eks. elev og forælder. Dette for at guide brugeren til oprettelsesprocessen.

Vi fulgte den 8. heuristik Aesthetic and minimalist design, ved at holde vores design minimalistisk, og undgå unødvendige elementer og information, som potentielt kan forvirre og distrahere brugeren.

Vi fulgte 9. heuristik, da vi har lavet alle inputfelter/forms obligatoriske ved CRUD opretning og redigering. Når slutbrugeren undlader at indtaste nødvendige oplysninger, eller indtaster forkerte oplysninger, ville de modtage en generel fejlbesked.

Mockup prototype (udarbejdet i fællesskab, beskrevet af Maria, Frederik og Kasper)

Vi brugte Figma til at lave en mockup prototype af den endelige hjemmeside. Link til Figma findes [her](#). Til udarbejdelsen af vores prototype brugte vi især Gestaltlovene¹².



Forside (Maria):

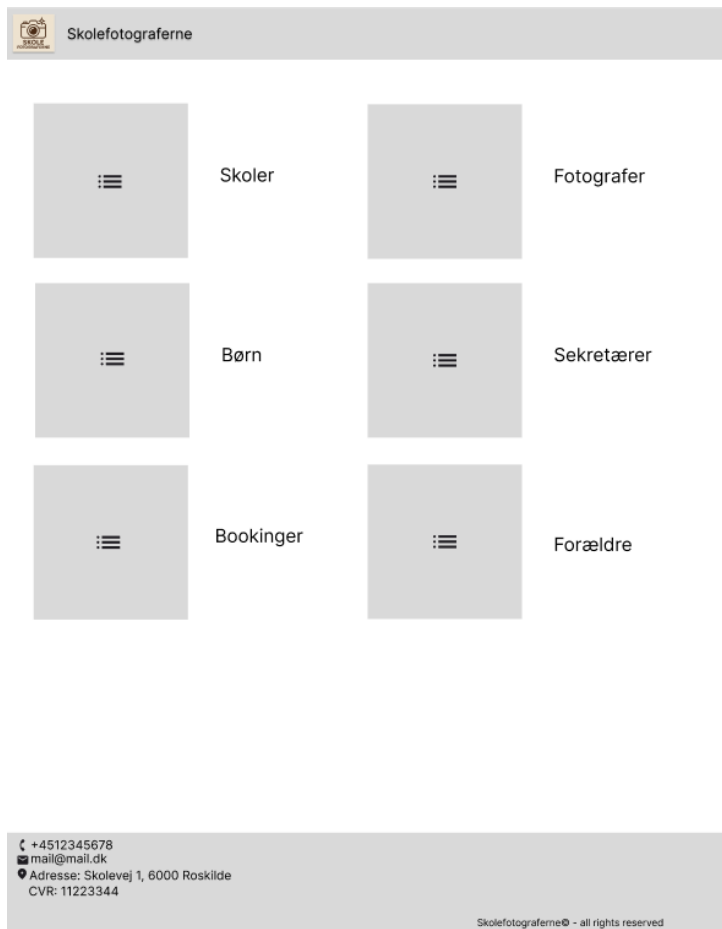
Til vores forsidedesign har vi især benyttet *loven om nærhed*, og placeret elementer der hører sammen i nærheden af hinanden. Det ses bl.a. i vores navigation, hvor undersiderne er placeret tæt sammen i højre side, men også i vores logo, som er placeret i venstre side med en tilhørende tekst.

Vi har også benyttet *loven om ensartethed* ved at gøre alle elementer på forsiden samme form.

I vores navigation bruger vi ligeledes *loven om forbundethed* og *loven om Common Region*, idet vores navigation

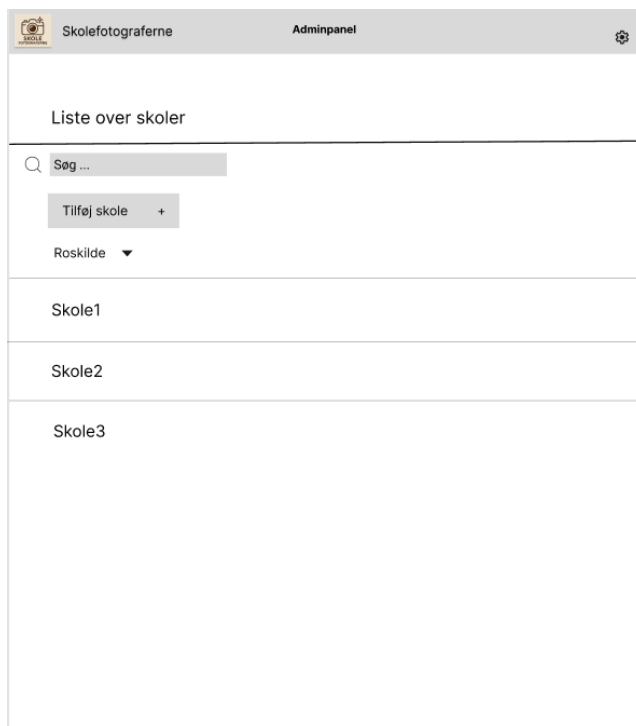
er omkranset af en boks. Dette for at brugeren ved, at disse elementer hører sammen.

¹² (Prohazka & Buus, n.d.)



Admin-oversigt (Kasper):

Til vores Adminoversigt, har vi benyttet *Loven om ensartethed* da alle boksene på siden er ens og alle giver adgang for admin til diverse sider om de forskellige typer roller i systemet. Vi har også lagt vægt på *Loven om Forbundethed* da de grå bokse tydeligt referere over til teksten fra højre af dem.



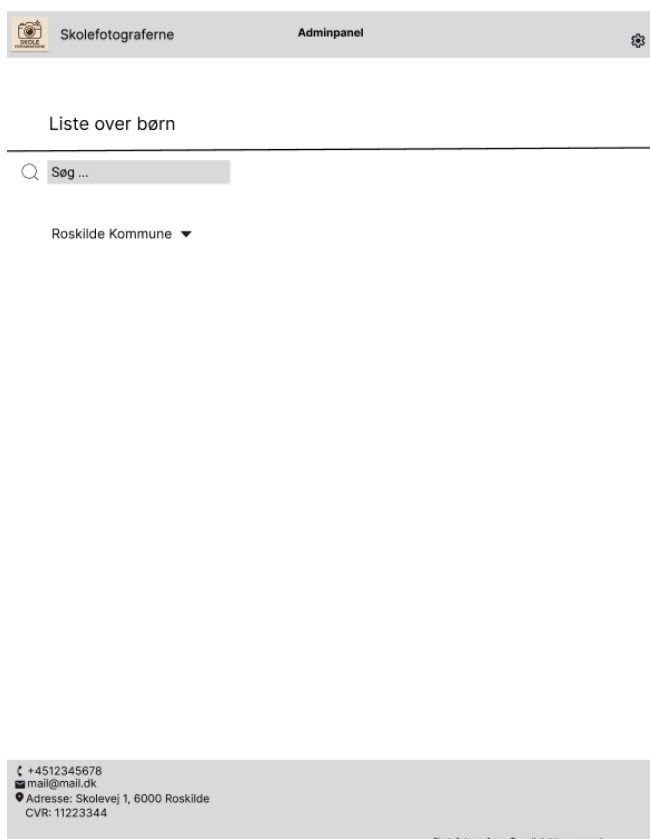
Side til skoleoversigt (Frederik):

Ideén var at have individuelle oversigter over hver enkel skole ud fra en kommune.

Der skulle også kunne tilføjes en ny skole til systemet, samt filtreres eksisterende skoler i systemet. Denne oversigtsside var tiltænkt at skulle benyttes af sidens admin(s).

Vi har benyttet 'loven om common region' ved at separere skoler ud fra

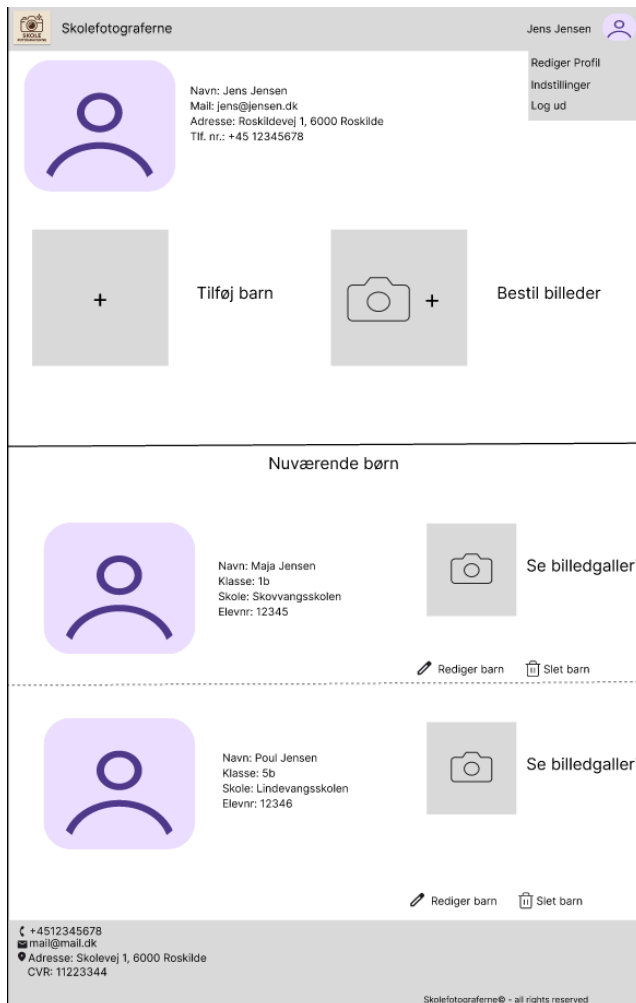
kommuner i et dropdown, samt lavet seperingslinjer efter hver skole.



Side til elevoversigt (Frederik):

Ideén var at have individuelle oversigter over hver enkel elev ud fra en kommune samt at kunne filtrere elever i søgefeltet. Siden var tiltænkt til sidens admin(s).

Her har vi også benyttet 'loven om common region' ved at separere elever ud fra kommuner i et dropdown.



Forældreside (Frederik og Maria):

Forældresiden skulle have funktionalitet i form af at kunne oprette deres barn i systemet, samt bestille fotos taget af deres barn. De skulle også kunne redigere eller slette barnet fra systemet. Herudover skulle forældrene have en oversigt over deres nuværende børn i systemet, hvor de skulle kunne tilgå et billedgalleri med billeder tilhørende barnet.

Til denne underside har vi ligeledes brugt Gestaltlovene.

Vi har brugt *loven om figurdannelse*, idet vi har brugt linjer (både fuldoptrukne og stiplede) til at inddеле elementer, som hører sammen.

Igen har vi også brugt *loven om nærhed* ved at placere elementer, der hører sammen, tæt på hinanden, fx billede og billedetekst.

Udviklingsproces

Scrum (Maria)

Undervejs i vores projekt har vi brugt den agile arbejdsmetode Scrum¹³ til at planlægge og holde styr på vores arbejdsforløb. Vi har opdelt arbejdet i 0-4 såkaldte Sprints, hvor vi inden hvert sprint har holdt Sprint Planning af kommende sprint, og i slutningen af hvert sprint har afholdt Sprint Review og Sprint Retrospective. Hver morgen har vi desuden afholdt Daily Scrum af 15 minutters varighed, hvor vi har lagt en plan for dagen og snakket om, hvad vi har lavet siden sidst. Vores Daily Scrums kan ses i bilag 1, og en planlægning over vores Scrum Events kan ses i bilag 4.

Sprint 0 - uge 12-18

Sprint Planning (16/03/2026) (fælles)

Sprint Goal:

- At komme godt i gang med projektet og de indledende faser
- Få lavet nogle indbyrdes aftaler og en fælles gruppekontrakt
- Fremvisning af Figma prototype for PO
- Stille PO indledende og afklarende spørgsmål til projektet
- At få lavet virksomhedsanalyse herunder:
 - BMGC
 - SWOTT
 - Inception Deck
- Userstories og estimering af userstories
- Gantt-kort

¹³ (Schwaber & Sutherland, 2020)

- Domænemodel
- ER-diagram
- Gået i gang med klassesdiagram
- Opsætning af projekt på GitHub og i Visual Studio

Sprint review (01/05/2026) (Frederik, Kasper og Maria)

Vi startede med at lave Business Model Generation Canvas, hvor vi bl.a. har analyseret kundesegmenter, kunderelationer, indtægtskilder og nøgleressourcer for Skolefotograferne.

16. april præsenterede vi et mockup lavet i Figma af vores hjemmeside til PO Vicki. Mock Up'en skulle fungere som vores første prototype af vores nye system for Skolefotograferne, og vi fik umiddelbart god feedback fra Vicki, men også enkelte ideer til forbedringer og udvidelser.

Vi har lavet SWOT analyse af Skolefotograferne, og undersøgt styrker/svagheder og muligheder/trusler for virksomheden.

Vi har lavet Inception Deck, hvor vi har besvaret spørgsmålene "Hvorfor er vi her?", "Hvad skal vi IKKE lave i projektet?", "Hvem er interessenterne?" og "Hvad holder os vågne om natten?" Vi har desuden lavet et Elevator-pitch og en Product Box.

Vi gik videre til at lave domænemodel i Draw.io og userstories opsat i Trello med Acceptance Criteria og Tasks.

Vi påbegyndte estimering af vores udarbejdede userstories ved brug af triangulation, samt lavede Burndown-chart, hvor vi talte alle vores story matrix points sammen og fordelte dem ud på alle vores sprints. Dette beregnede vi ved at plusse alle userstory point og dividere dem med antallet af vores sprint. Vi lavede vores Burndown-chart over både vores planlagte arbejdsproces, samt en til den aktuelle arbejdsproces over userstory points.

Vi lavede Gantt-kort¹⁴ for at visualisere hele arbejdsprocessen over tid for projektet, og fordelte vores “epics” i sprints.

Vi startede med vores design-klassediagram, som udgør ‘skelettet’ af vores system.

Vi har lavet vores ER-diagram ud fra vores domæne model, som senere skal blive til vores database.

Vi fik opsat projektet i Visual Studio og hentet relevante NuGet packages/dependencies.

Vi fik forbundet vores individuelle Simply.com databaser til kodebasen.

Vi fik opsat GitHub repository til kodedelen af projektet.

Da vi følte, vi var godt med i forhold til vores virksomhedsanalyse og indledende faser, gik vi så småt i gang med at programmere, før vi egentlig havde planlagt. Vi fik skitseret vores modelklasser, og snakket og diskuteret opsætningen af hele vores program: hvordan vil vi opsætte vores repositories? Vil vi bruge interfaces eller arv? Hvilke klasser skal være abstrakte? Osv.

Desuden har vi diskuteret projektets scope, og hvad der er realistisk at nå i den begrænsede tid, vi har. Vi har af den grund valgt at skære nogle funktioner fra, som egentlig skulle have været en del af den færdige prototype, herunder private fotosessions og skolelærerens mulighed for at booke.

Sprint retrospective (01/05/2026) (Frederik, Kasper og Maria)

Hvad fungerede godt?

- Vi har en udmærket gruppedynamik, og er gode til at samarbejde og kommunikere
- Vi er effektive og arbejdsomme, når vi først er i gang
- Vi har nået de ting vi skulle have lavet i dette sprint

¹⁴ Se bilag 3

Hvad fungerede mindre godt?

- Ikke alle er lige gode til at give besked om fravær/når de kommer for sent
- Vi har lidt kastet os ud i projektet uden en plan og har nogle dage manglet overblik
- Vi kommer ofte til at snakke i munden på hinanden
- Vi kommer ofte til at gå i gang med flere ting på én gang
- Mindre uenigheder om design af koden

Hvilke konkrete tiltag kan forbedre næste sprint?

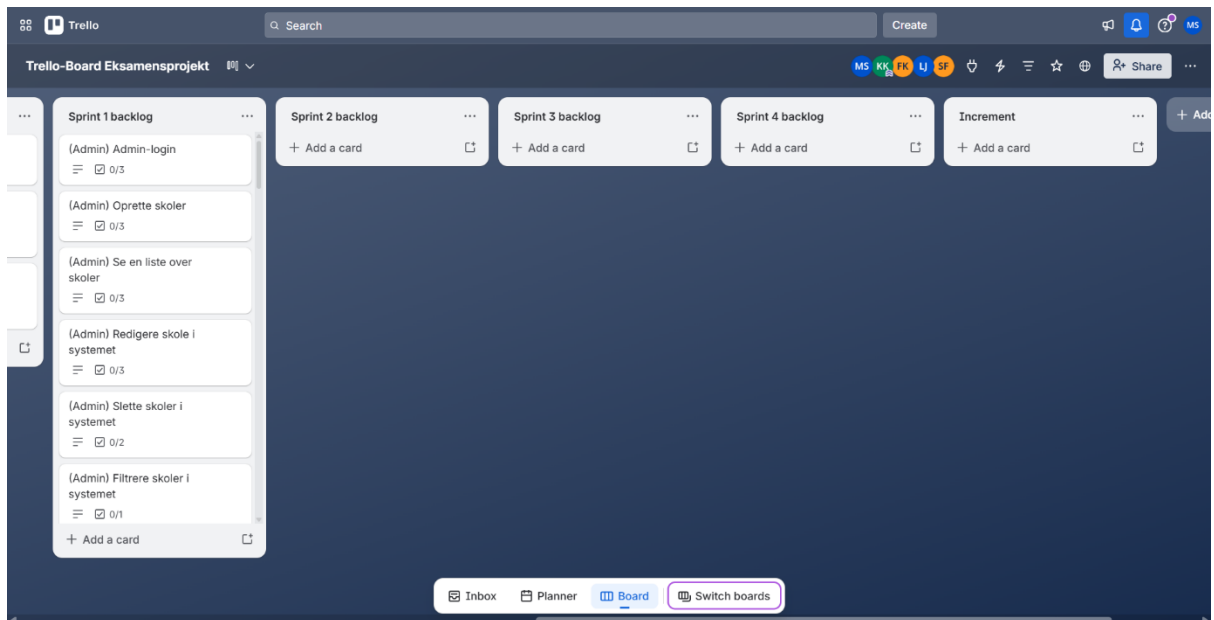
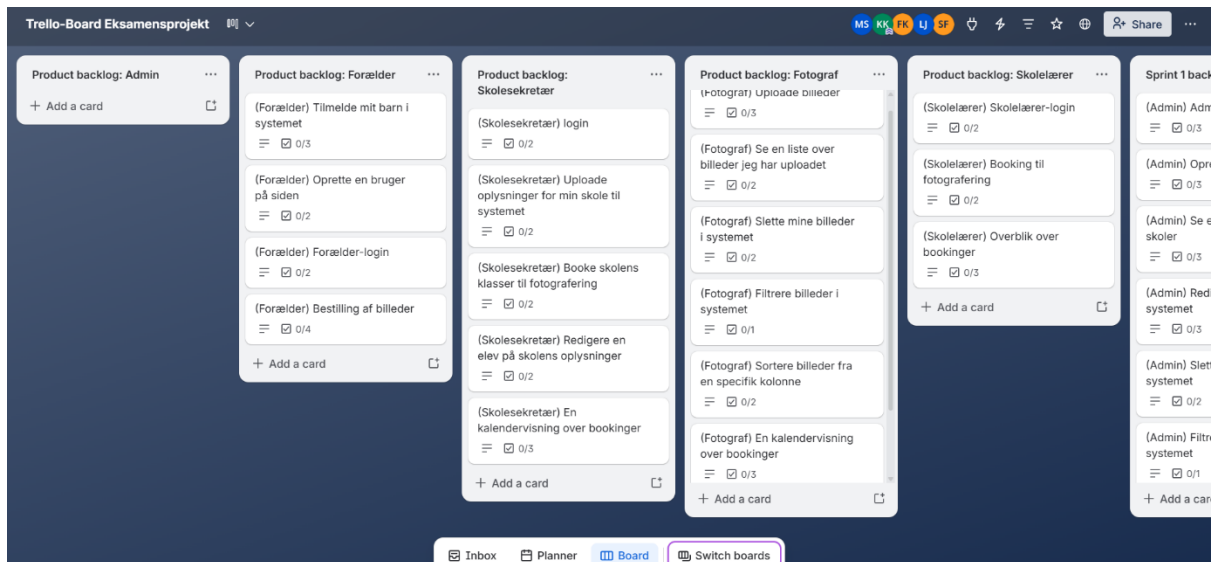
- Være lidt bedre til at planlægge og følge planen
- Lad folk snakke ud, og undgå at afbryde
- Giv besked ved fravær
- Prøve så vidt som muligt at ligeligt fordele arbejdsopgaver
- Være åbne overfor forslag, men også realistiske i forhold til hvad vi kan nå og hvad vi har lært op til nu og følge pensum
- I den perfekte verden, vil det være bedst, hvis vi alle kunne møde op, alle dage

Sprint 1 - uge 19

Sprint Planning (01/05/2026) (fælles)

Sprint Goal:

- Starte med at få uddelt arbejdsopgaver (userstories) og få prioriteret vores userstories
- Lav SQL script til at oprette tabeller i databasen



Ovenfor ses et udsnit af vores Trelloboard og vores Product Backlog. Vi opdelte vores userstories i kategorierne Admin, Forælder, Skolesekretær, Fotograf, Skolelærer, fordi det er de 5 roller der findes i systemet. Herefter ses de 4 sprint backlogs, hvor vi løbende flyttede userstories over alt efter, hvordan vi planlagde at arbejde med dem. Senere tilføjede vi også en *Done* liste, til de userstories som var færdigimplementeret, men endnu ikke helt klar til at komme over i *Increment*.

Sprint backlog og arbejdsproces (Maria)

Vi startede med at lave vores SQL scripts og SchoolRepositoryAsync i fællesskab, så vi alle havde et fælles udgangspunkt for at gå videre med at arbejde med vores CRUD- og filtreringsmetoder for de resterende admin klasser.

Derefter fordelte vi vores userstories, så vi hver især har ansvar for to repositories og de dertilhørende metoder. Se de fulde userstories i bilag.

Frederik:

- Admin userstories omhandlende Photographer og PhotoEvent

Kasper:

- Admin userstories omhandlende SchoolClass og Student

Lukas:

- Admin userstories omhandlende Teacher og OrderLine

Maria:

- Admin userstories omhandlende Photo og Parent

Sinan:

- Admin userstories omhandlende Order og Secretary

Kodeeksempler

Lukas:

```
public async Task<List<Teacher>> FilterAsync(string filterCriteria)
{
    string query = @"
        SELECT * FROM Teacher
        WHERE TeacherID LIKE @filterCriteria
        OR TeacherName LIKE @filterCriteria
        OR Mail LIKE @filterCriteria
        OR PhoneNumber LIKE @filterCriteria";
    List<Teacher> teachers = new List<Teacher>();
    using (SqlConnection connection = new SqlConnection(Secret.ConnectionString))
    {
        try
        {
            await connection.OpenAsync();
            SqlCommand command = new SqlCommand(query, connection);
            command.Parameters.AddWithValue("@filterCriteria", $"{filterCriteria}%");
            SqlDataReader reader = await command.ExecuteReaderAsync();
            while (await reader.ReadAsync())
            {
                int teacherID = reader.GetInt32("TeacherID");
                string mail = reader.GetString("Mail");
                string teacherPassword = reader.GetString("TeacherPassword");
                string teacherName = reader.GetString("TeacherName");
                string phoneNumber = reader.GetString("PhoneNumber");
                Teacher teacher = new Teacher(id:teacherID, mail:mail, name:teacherName, password:teacherPassword, phoneNumber:phoneNumber);
                teachers.Add(teacher);
            }
            reader.CloseAsync();
        }
        catch (SqlException sEx)
        {
            sEx.PrintWithType();
            throw new RepositoryException(RepositoryExceptionType.Read, "Kan ikke indlæses");
        }
        catch (Exception ex)
        {
            ex.PrintWithType();
            throw new RepositoryException(RepositoryExceptionType.Read, ex.GetFullMessage());
        }
    }
    return teachers;
}
```

Ovenstående metode bruges til at filtrere alle 'Teachers', som har en eller flere værdier, der matcher med 'filterCriteria'.

De bliver matchet i SQL via 'Like' statements og til 'filterCriteria' tilføjes '%' før og efter så der bliver tjekket om attributten indeholder 'filterCriteria'. SQL querien finder alle de rette 'Teachers'. Derefter hentes de forskellige værdier fra de fundne databaserækker og sættes i lokale variabler via 'reader.GetString' fx.

Til sidst lægges værdierne i et nyt 'Teacher' objekt og returneres som en liste.

Frederik:

```
public async Task CreateAsync(Photographer photographer)
{
    string query =
        @"INSERT INTO
        Photographer(PhotographerName, Mail, PhoneNumber, CVR, PhotographerPassword)
        Values(@PhotographerName, @Mail, @PhoneNumber, @CVR, @PhotographerPassword)";
    try
    {
        await using SqlConnection connection = new(Secret.ConnectionString);
        await connection.OpenAsync();
        await using SqlCommand command = new(query, connection);
        command.Parameters.AddWithValue("@PhotographerName", photographer.Name);
        command.Parameters.AddWithValue("@Mail", photographer.Mail);
        command.Parameters.AddWithValue("@PhoneNumber", photographer.PhoneNumber);
        command.Parameters.AddWithValue("@CVR", photographer.CVR);
        command.Parameters.AddWithValue("@PhotographerPassword", photographer.Password);
        await command.ExecuteNonQuery();
    }
    catch (SqlException e)
    {
        e.PrintWithType();
        throw new RepositoryException(RepositoryExceptionType.Create, "Ugyldigt input");
    }
    catch (Exception e)
    {
        e.PrintWithType();
        throw new RepositoryException(RepositoryExceptionType.Create, e.GetFullMessage());
    }
}
```

Eksempel på CreateAsync metode i PhotographerRepositoryAsync.

Metoden står for at oprette en fotograf i systemet. Metoden tager imod en fotograf som argument, som skal oprettes i databasen. Metoden binder fotografens relevante properties i koden med SQL attributterne, der giver mening ift. vores ER diagram, for så at kunne indsætte dem igennem en query forespørgsel. Metoden opfanger SqlExceptions, og vil i så fald kaste en RepositoryException af typen RepositoryExceptionType.Create.

Kasper:

```
6 references | KasperKronmannMoeller, 2 hours ago | 2 authors, 2 changes
public async Task DeleteAsync(SchoolClass item)
{
    string query = "DELETE FROM SchoolClass WHERE SchoolClassID = @SchoolClassID";
    using (SqlConnection connection = new SqlConnection(Secret.ConnectionString))
    {
        try
        {
            SqlCommand command = new SqlCommand(query, connection);
            await connection.OpenAsync();
            command.Parameters.AddWithValue("@SchoolClassID", item.Id);
            await command.ExecuteNonQuery();
        }
        catch (SqlException sEx)
        {
            sEx.PrintWithType();
            throw new RepositoryException(RepositoryExceptionType.Delete, "Ugyldig ID");
        }
        catch (Exception ex)
        {
            ex.PrintWithType();
            throw new RepositoryException(RepositoryExceptionType.Delete, ex.GetFullMessage());
        }
    }
}
```

Ovenstående viser metoden DeleteAsync, denne metode har et item af typen SchoolClass som parameter og returtypen er af typen Task. Metoden har en string query, som udvælger den Skoleklasse med det identiske ID og eksekverer metoden så skoleklasse bliver slettet fra Database, som den kender til via min ConnectionString fra klassen Secret. Hvis den ikke finder nogle skoleklasser med et gyldigt ID, smider metoden en Exception, med teksten "Ugyldig ID".

Maria:

```

10 references | Maria Litske Strauss, 1 day ago | 2 authors, 2 changes
145 public async Task<List<Parent>> GetAllAsync()
146 {
147     string query = "SELECT * FROM Parent";
148     List<Parent> parents = new List<Parent>();
149     using (SqlConnection connection = new SqlConnection(Secret.ConnectionString))
150     {
151         try
152         {
153             await connection.OpenAsync();
154             SqlCommand command = new SqlCommand(query, connection);
155             SqlDataReader reader = await command.ExecuteReaderAsync();
156             while (await reader.ReadAsync())
157             {
158                 int parentId = reader.GetInt32("ParentID");
159                 string parentName = reader.GetString("ParentName");
160                 string mail = reader.GetString("Mail");
161                 string phoneNumber = reader.GetString("PhoneNumber");
162                 string parentAddress = reader.GetString("ParentAddress");
163                 string parentPassword = reader.GetString("ParentPassword");
164                 string postalCode = reader.GetString("PostalCode");
165                 Parent parent = new Parent(parentId, mail, parentPassword, parentName, phoneNumber, parentAddress, postalCode);
166                 parents.Add(parent);
167             }
168             reader.CloseAsync();
169         }
170         catch (SqlException sEx)
171         {
172             sEx.PrintWithType();
173             throw new RepositoryException(RepositoryExceptionType.Read, "Kan ikke indlæses");
174         }
175         catch (Exception ex)
176         {
177             //ExceptionHelpers.PrintWithType(ex);
178             ex.PrintWithType();
179             throw new RepositoryException(RepositoryExceptionType.Read, ex.GetFullMessage());
180         }
181     }
182     return parents;
183 }

```

Ovenstående metode er fra ParentRepositoryAsync-klassen, hvis formål er at hente alle forældre-objekter og deres tilhørende oplysninger fra databasen, så frontend-delen kan håndtere det. Dette gøres via en SQL streng "SELECT * FROM Parent", som selecter al information om alle forældre. Derefter laves der en liste 'parents', som skal indeholde de forældreobjekter, som senere skal vises i listen. Denne efterfølges af et Sql connection objekt, som skaber og åbner forbindelsen til databasen. Connection objektet skal fjernes igen, og det gøres med et Using statement, fordi det automatisk fjerner connectionen, når scopet er færdigt. Herefter åbnes connectionen med .OpenAsync, og der laves et nyt SqlCommand objekt, som tager imod vores query string og et connection objekt.

Derefter eksekveres command objektet og én række data fra databasen læses ad gangen med Read(). Så skabes et nyt parent objekt med de værdier, som er blevet læst ind, og parent objektet tilføjes til vores liste 'parents'. Til sidst lukkes readeren, og listen af parents returneres.

Sinan:

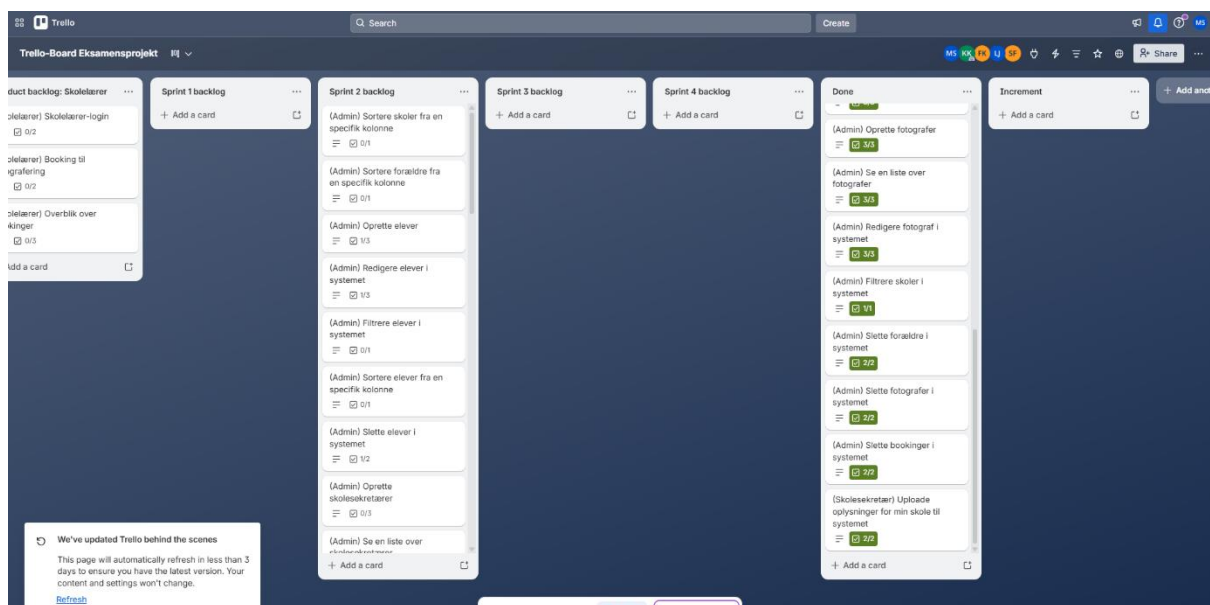
```
public async Task UpdateAsync(Order item)
{
    string query = "UPDATE PhotoOrder SET PhotoOrderDate = @PhotoOrderDate, TotalPrice = @TotalPrice, ParentID = @ParentID, PhotoOrderID = @PhotoOrderID";
    using(SqlConnection connection = new SqlConnection(Secret.ConnectionString))
    {
        try
        {
            await connection.OpenAsync();
            SqlCommand command = new SqlCommand(query, connection);
            command.Parameters.AddWithValue("@PhotoOrderID", item.Id);
            command.Parameters.AddWithValue("@PhotoOrderDate", item.Date);
            command.Parameters.AddWithValue("@TotalPrice", item.TotalPrice);
            command.Parameters.AddWithValue("@ParentID", item.Parent);
            await command.ExecuteNonQueryAsync();
        }
        catch (SqlException e)
        {
            e.PrintWithType();
            throw new RepositoryException(RepositoryExceptionType.Update, "Ugyldigt input");
        }
    }
}
```

Billedet starter med et query som er en sql statement for at opdater hver rows i hver kolonne i databasen. Under Try åbner vi først op for databasen som vi instanceret med SqlConnection, derefter bruges parameteren til at finde værdierne i rows og kolonnerne, så de kan derefter blive opdateret med de nye værdier via ExecuteNonQueryAsync(). Til sidst er der en exception, hvis der kommer en fejl som et ugyldigt input.

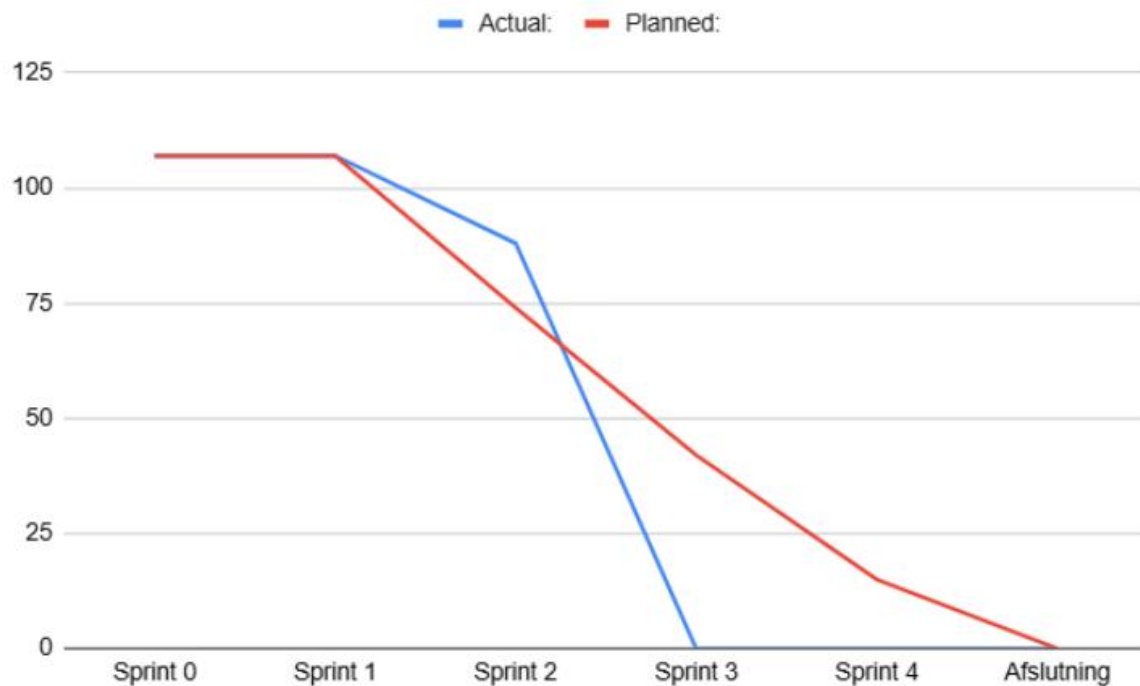
Sprint review (08/05/2026) (fælles)

- Vi startede med at lave SQL scripts for at kunne indsætte data ind i databasen.
- Vi lavede et generisk IRepository interface, som alle repositories nu implementerer.
- Vi fordelte userstories ud imellem os, så vi hver især havde ansvar for 2 repositories
- Vi har nået at lave følgende userstories (done):
 - Admin - oprette skoler
 - Admin - se liste over skoler
 - Admin - redigere skoler
 - Admin - slette skoler
 - Admin - filtrere skoler
 - Admin - oprette forældre

- Admin - se liste over forældre
- Admin - redigere forældre
- Admin - slette forældre
- Admin - filtrere forældre
- Admin - oprette fotografier
- Admin - redigere fotografier
- Admin - slette fotografier
- Admin - slette fotoevents



Trello scrum-board slutningen af sprint 1



Burndown chart for sprint 1

Sprint retrospective (08/05/2026)(fælles)

Hvad fungerede godt?

- God fordeling af arbejdsbyrden
- God kommunikation
- Gode til at hjælpe hinanden
- Gode til at overholde Daily Scrum og Scrum generelt

Hvad fungerede mindre godt?

- Komplikationer ved online møde og kommunikation
- Snakke i munden på hinanden
- Tekniske problemer og udfordringer
- Mødeustabilitet

- Manglende kommunikation ved fravær
- Sygdom
- Svært at få hjælp af Poul
- Svært at komme videre, hvis man sidder fast i et problem

Hvilke konkrete tiltag kan forbedre næste sprint?

- Lave flere ting i fællesskab, når vi ikke selv kan løse problemerne

Hvad har vi lært i dette sprint?

- Der er et stort strukturelt arbejde i at arbejde med userstories, og det er en meget tidskrævende proces
- Vi har undervejs fundet ud af, at vi manglede nogle userstories, og har også måtte spørge os selv "Hvem skal denne userstory omhandle? "Hvem skal kunne XYZ?"
- Vi har fundet ud af, at vores userstories skulle have været mere opdelt ift. backend/frontend. Vi fandt ud af undervejs, at vi ikke kunne markere vores userstories som 'Done', fordi vi både har frontend/backend i vores userstories, men vi i vores arbejdsproces valgte at starte med backend.
- Vi endte med at have gentagne userstories, der omfattede samme funktionalitet ift. CRUD
- Vi opdagede også, at nogle userstories ikke giver mening fx CreateOrder og UpdateOrder

Sprint 2 - uge 20

Sprint Planning (08/05/2026) (fælles)

Sprint Goal:

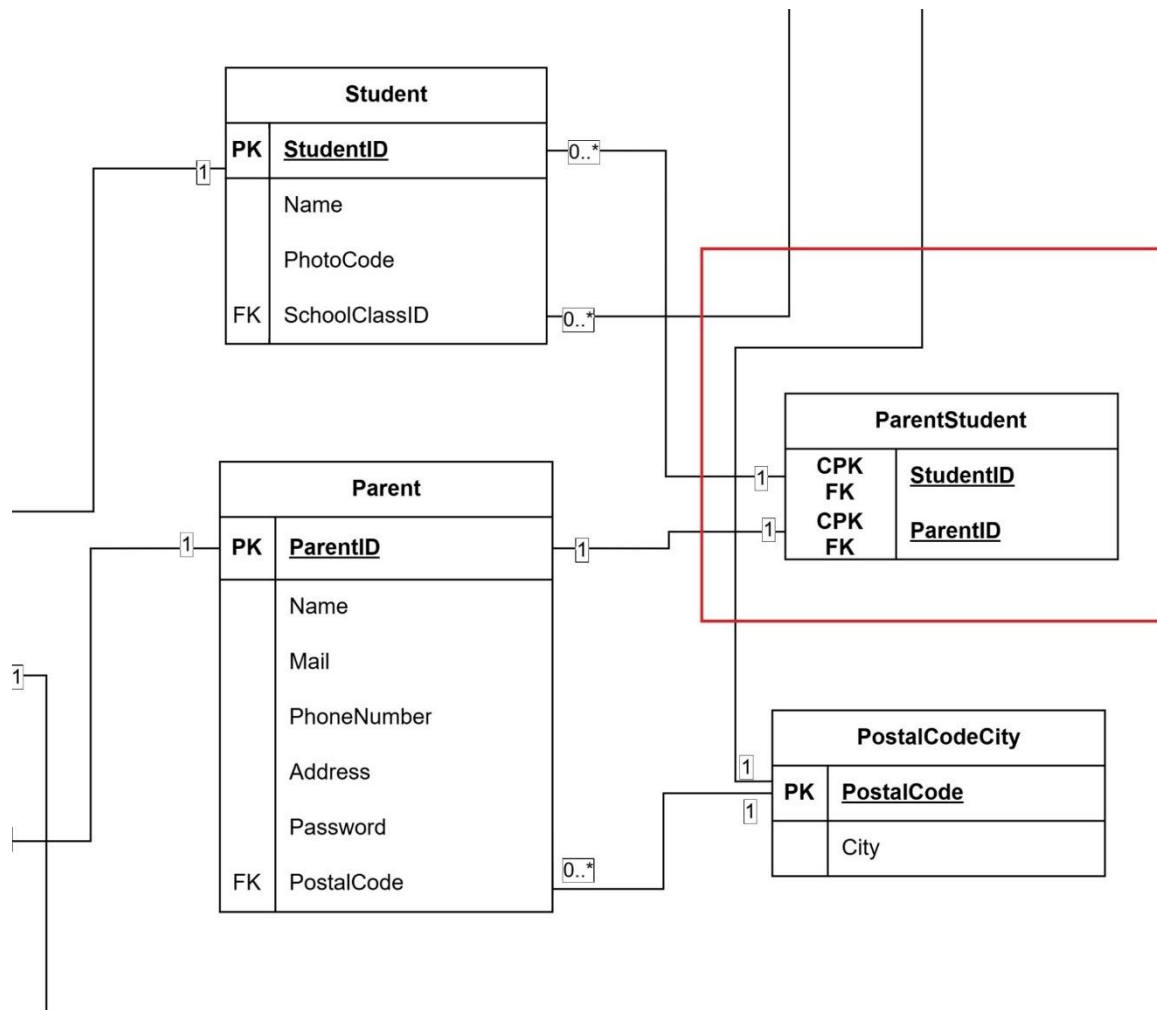
- Færdiggøre CRUD vi er i gang med (rette fejl på metoder som ikke fungerer)
- Implementere/tænke over design på razorpages Måske?
- Opdatere klassesdiagram (sørge for at det stemmer overens med de ændringer, vi har lavet i koden)
- Evt. begynde at sætte rapporten sammen

Sprint arbejdsproces (Maria)

Sprint 2 var kendetegnet ved en del frustration og en følelse hos nogle af os af, at projektet blev for uoverskueligt og stort at håndtere med for mange løse ender. Vi valgte derfor i sprint 2 at fokusere vores energi på vores CRUD-funktioner, og vente eller helt skære nogle funktioner fra.

Vi begyndte implementeringen af userlogin og fandt i den proces ud af, at vi nok havde brug for en separat user-tabel i vores database. Det var dog ret sent i vores arbejdsproces, og ville bevirke, at vi skulle ændre i en masse andre tabeller og kode. Vi valgte derfor at vente lidt med denne funktion.

Vicki ville gerne have muligheden for at tilføje flere forældre til et barn, og samtidig at en forældre kan have flere børn. Denne problematik forsøgte vi at løse med en ParentStudent tabel, fordi man ifølge 1NF ikke må have flere værdier og lister i én kolonne. På grund af tidspres og kompleksitet valgte vi dog i sprint 2 at ændre i vores tabel, så en elev kun kan have én forælder tilknyttet, da det andet gav os for mange problemer.



Udsnit af vores ER-diagram før vi fjernede ParentStudent tabellen

Samme problematik havde vi med Ordre, som vi forsøgte at løse med en OrdreLinje-tabel. Grundet kompleksiteten af ordre klassen, dens metoder og mange forbindelser til andre klasser, valgte vi dog at vente med denne funktion til senere, hvis vi har tid til det, og i stedet fokusere på vores andre klasser.

Kodeeksempler

Maria:

Opret Foto

Filnavn

Dato

01/01/0001 00.00

Fotograf

Vælg fotograf

Elev

Vælg elev

Vælg elev

Emil Jensen - 3A - 2024

Maja Christensen - 5B - 2024

```
public class CreatePhotoModel : PageModel
{
    private IRepositoryAsync<Photo> _repo;
    private IRepositoryAsync<Photographer> _photographerRepo;
    private IRepositoryAsync<Student> _studentRepo;

    [BindProperty]
    9 references | Maria Litske Strauss, 6 days ago | 1 author, 1 change
    public Photo NewPhoto { get; set; }

    [BindProperty]
    [Required]
    3 references | Maria Litske Strauss, 22 hours ago | 1 author, 2 changes
    public string PhotographerId { get; set; }

    [BindProperty]
    [Required]
    3 references | Maria Litske Strauss, 6 days ago | 1 author, 1 change
    public string StudentId { get; set; }

    [BindProperty]
    2 references | Maria Litske Strauss, 22 hours ago | 1 author, 1 change
    public IEnumerable<SelectListItem> PhotographerSelect { get; set; }
    [BindProperty]
    2 references | 0 changes | 0 authors, 0 changes
    public IEnumerable<SelectListItem> StudentSelect { get; set; }
```

```
2 references | Maria Litske Strauss, 22 hours ago | 1 author, 2 changes
public async Task OnGet()
{
    NewPhoto = new Photo();

    List<Photographer> photographers = await _photographerRepo.GetAllAsync();
    PhotographerSelect = photographers.Select(p => new SelectListItem { Value = Convert.ToString(p.Id), Text = $"{p.Name} - {p.Mail}" });
    List<Student> students = await _studentRepo.GetAllAsync();
    StudentSelect = students.Select(p => new SelectListItem { Value = Convert.ToString(p.Id), Text = $"{p.Name} - {p.SchoolClass.Name} - {p.SchoolClass.Year}" });
}

0 references | Maria Litske Strauss, 22 hours ago | 1 author, 5 changes
public async Task<IActionResult> OnPost()
{
    if (!ModelState.IsValid)
    {
        await OnGet();
        return Page();
    }
    try
    {
        NewPhoto.Photographer = await _photographerRepo.GetAsync(Convert.ToInt32(PhotographerId));
        NewPhoto.Student = await _studentRepo.GetAsync(Convert.ToInt32(StudentId));
        await _repo.CreateAsync(NewPhoto);
        return RedirectToPage("Index");
    }
}
```

```
<div class="form-group">
  <label asp-for="@Model.PhotographerId" class="control-label">Fotograf</label>
  <select asp-for="@Model.PhotographerId" asp-items="@Model.PhotographerSelect" id="PhotographerSelect" class="form-select"><option value="">Vælg fotograf</option></select>
</div>

<div class="form-group">
  <label asp-for="@Model.StudentId" class="control-label">Elev</label>
  <select asp-for="@Model.StudentId" asp-items="@Model.StudentSelect" id="StudentSelect" class="form-select"><option value="">Vælg elev</option></select>
</div>
```

Ovenstående kodeeksempel er fra CreatePhoto klassen, og viser hvordan vi implementerede en dropdown menu til at vælge hhv. eleven, som er på billedet, og fotografen, som har taget billedet.

Dette gjorde vi med to properties, PhotographerId og StudentId, begge af typen string, og to properties PhotographerSelect og StudentSelect af typen IEnumerable<SelectListItem> i den bagvedliggende cshtml.cs side.

På den foranliggende cshtml side bindes vores valgte værdi (fotograf) fra dropdown-menuen til vores property PhotographerId, når OnPost metoden kaldes.

PhotographerSelect er den property, som indeholder vores liste af valgmuligheder (fotografer).

Vores OnGet metode fylder vores dropdown-menu med værdier, og starter altså først med at hente vores liste af fotografer ned fra databasen. Herefter laves hver fotograf om til et SelectListItem, valuen sættes til ID og via stringinterpolation får vi skrevet værdierne navn og mail ud for at gøre det nemmere for brugeren.

I OnPost metoden tjekker vi først, om der er valgt en værdi. Hvis ikke, udskrives en fejl og vi returneres til siden og vores dropdown-menu fyldes igen. I try-delen henter vi vores valgte objekter ned fra databasen og sætter dem ind i NewPhoto propertyen via GetAsync metoden. Til sidst gemmes de i databasen via CreateAsync metoden.

Kasper:

Søg:

ID	Navn	Fotokode	Skoleklasse	Forælder	Slet Redigér
1	Emil Jensen	FOTO001	3A	Lars Jensen	Slet elev Redigér elev

Liste over elever

Opret ny elev

Søg: Alle Filtrér

ID	Navn	Fotokode	Skoleklasse	Forælder
1	Emil Jensen	OTO001	3A	Lars Jensen
2	Maja Christensen	OTO002	5B	Pia Christensen

```
public class IndexModel : PageModel
{
    #region Instance fields
    /// <summary>
    /// Instance field that represents the repository for the Student model.
    /// Used to access the data for the students in the system.
    /// It is injected into the constructor of the IndexModel class and is used in the OnGet method to retrieve all students from
    /// </summary>
    private IRepositoryAsync<Student> _repo;
    #endregion

    #region Properties
    /// <summary>
    /// Property that represents the list of students that will be displayed on the index page.
    /// Used to store the students that are retrieved from the database and filtered and sorted based on user input.
    /// </summary>
    12 references | KasperKronmannMoeller, 3 days ago | 1 author, 2 changes
    public List<Student> Students { get; set; }
    /// <summary>
    /// Property that represents the filter criteria entered by the user in the search box on the index page.
    /// Used to filter the list of students based on the user's input.
    /// </summary>
    [BindProperty(SupportsGet = true)]
    11 references | KasperKronmannMoeller, 3 days ago | 1 author, 2 changes
    public string FilterCriteria { get; set; }
    /// <summary>
    /// Property that represents the filter option selected by the user in the dropdown menu on the index page.
    /// Used to determine which property of the Student model to filter by based on the user's selection.
    /// </summary>
    [BindProperty(SupportsGet = true)]
    3 references | KasperKronmannMoeller, 3 days ago | 1 author, 2 changes
    public string FilterBy { get; set; }
}
```

```
public async Task OnGet()
{
    var allStudents = await _repo.GetAllAsync();

    if (!string.IsNullOrEmpty(FilterCriteria))
    {
        switch (FilterBy)
        {
            case "StudentName":
                Students = allStudents
                    .Where(sc => sc.Name.Contains(FilterCriteria, StringComparison.OrdinalIgnoreCase))
                    .ToList();
                break;

            case "PhotoCode":
                Students = allStudents
                    .Where(sc => sc.PhotoCode.ToString().Contains(FilterCriteria))
                    .ToList();
                break;

            case "SchoolClassID":
                Students = allStudents
                    .Where(sc => sc.SchoolClass.Name.Contains(FilterCriteria, StringComparison.OrdinalIgnoreCase))
                    .ToList();
                break;

            case "ParentID":
                Students = allStudents
                    .Where(sc => sc.Parent.Name.Contains(FilterCriteria, StringComparison.OrdinalIgnoreCase))
                    .ToList();
                break;

            case "All":
                Students = allStudents
                    .Where(sc =>
                        sc.Name.Contains(FilterCriteria, StringComparison.OrdinalIgnoreCase) ||
                        sc.PhotoCode.ToString().Contains(FilterCriteria) ||
                        sc.SchoolClass.Name.Contains(FilterCriteria, StringComparison.OrdinalIgnoreCase) ||
                        sc.Parent.Name.Contains(FilterCriteria, StringComparison.OrdinalIgnoreCase))
                    .ToList();
                break;
        }
    }
}
```

```
<form method="get">
  Søg: <input type="text" asp-for="@Model.FilterCriteria" />
  <select asp-for="@Model.FilterBy" class="form-select d-inline-block" style="width:auto; margin-left: 8px;">
    <option value="All">Alle</option>
    <option value="StudentName">Navn</option>
    <option value="PhotoCode">Fotokode</option>
    <option value="SchoolClassID">Skoleklasse</option>
    <option value="ParentID">Forælder</option>
  </select>
  <button type="submit" class="btn btn-success btn-sm">Filtrér</button>
</form>
```

Ovenstående kodeeksempel er fra Index-siden af Student-klassen. På billederne ses, hvordan vi har implementeret, så man kan filtrere en elev ud fra navn, fotokode, deres skoleklasses navn eller deres forældres navn. Dette har vi gjort ved at have to properties, FilterBy og FilterCriteria, begge to af typen string, samt en property kaldet Students, som er af typen List<Student>. Disse properties benyttes i OnGet-metoden, som bruger GetAllAsync til først at gemme alle students i en variabel.

Derefter tjekker metoden, i den første if-sætning, om FilterCriteria er null. Hvis den ikke er null, kører metoden videre til et switch statement, hvor vi benytter FilterBy-propertyen, som sikrer, at vi kan lave flere cases baseret på vores filtreringsønske. Der har vi en case for hver, hhv. StudentName, PhotoCode osv.

På den foranliggende side har vi valgt at lave en form, som sikrer, at brugeren kan vælge, hvilket filtreringsbehov brugeren ønsker.

På de to øverste billeder ses det tydeligt, hvordan man kan vælge filtreringsønske, og at hvis man vælger navn og søger efter 'Emil', får man kun elever med dette navn. Dette fungerer ligeledes, hvis man nu ønsker at søge efter efternavn også. Hvis man ønsker at kunne sortere, efter at man har filtreret, kan man også det, da sorteringen ikke nulstiller ens filtrering.

Sprint review (15/05/2026) (Maria og Kasper)

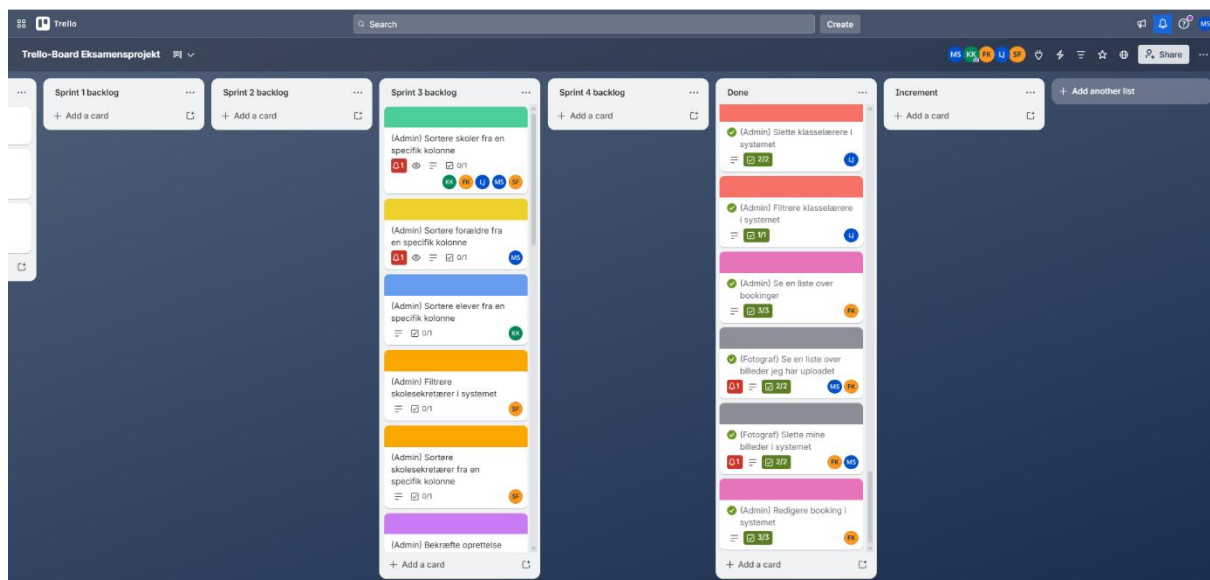
- Vi brugte en del tid de første to dage af sprintet på at forsøge at rette op på gennemgående fejl og løse ender i systemet, men uden held
- Vi begyndte på at implementere login-system, men dette er endnu ikke færdigimplementeret. Vi diskuterede også, om vi skulle have en separat user-tabel i vores database

- Vi begyndte med at implementere filtrering og validering på vores pages
- Vi fik tilføjet error messages på vores forms
- Vi begyndte at sætte vores rapport sammen
- Vi slettede vores ParentStudent tabel, og gjorde så forældre kun kan have ét barn og et barn kun kan have én forældre
- Vi valgte at sætte Order og OrderLine på pause, da vi vurderede, at de er for komplekse og kræver for meget tid, og i stedet valgte vi at fokusere vores energi på at få de andre funktionaliteter til at fungere
- Onsdag fik vi rettet op på de gennemgående fejl og løse ender på vores CRUD-funktioner og pages

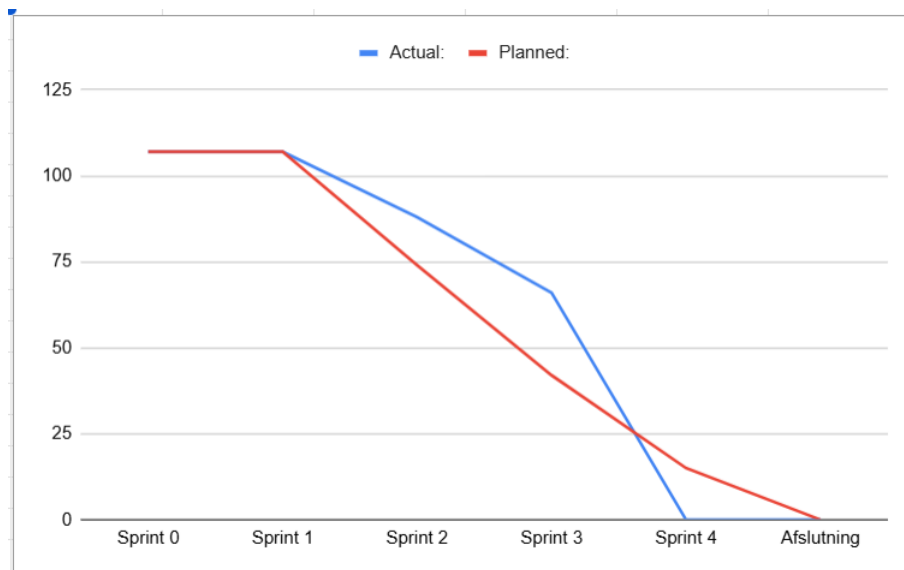
Vi har nået at lave følgende userstories (done):

- Admin - oprette elever
- Admin - redigere elever
- Admin - filtrere elever
- Admin - slette elever
- Admin - oprette skolesekretær
- Admin - se en liste over skolesekretærer
- Admin - rediger skolesekretærer
- Admin - slette skolesekretærer
- Admin - oprette fotograf
- Admin - filtrere fotografier
- Admin - oprette klasselærer
- Admin - se en liste over klasselærer

- Admin - rediger klasselærere
- Admin - slette klasselærere
- Admin - filtrere klasselærere
- Admin - se liste over bookinger
- Admin - se en liste over fotos
- Admin- slette fotos
- Admin - rediger booking



Trello scrum-board slutningen af sprint 2



Burndown chart for sprint 2

Sprint retrospective (15/05/2026) (fælles)

Hvad fungerede godt?

- Vi har været gode til at hjælpe hinanden
- Velfungerende samarbejde
- Vi fik rettet op på løse ender og føler nu, at vi er godt med rent tidsmæssigt
- Vi har fået afklaret med Poul hvad der er realistisk at få færdiggjort inden for vores begrænsede tidsramme for at færdiggøre projektet.

Hvad fungerede mindre godt?

- Fraværende gruppemedlemmer i starten af dette sprint
- Vi brugte lang tid på at udbedre fejl uden at komme nogen vegne

Hvilke konkrete tiltag kan forbedre næste sprint?

- Fortsætte med at være gode til at hjælpe hinanden

- Gå tidligt i seng, så man kan komme op om morgenen
- God søvnhygiejne!
- Husk at give besked i god tid, hvis man er sent på den/ikke kommer
- Arbejde til kl 14.00 i stedet for kl 13.00 nogle dage

Hvad har vi lært i dette sprint?

- Det er okay at arbejde i main branch og gemme på sin egen computer, men pas på med at pushe kode med FEJL!!!
- Fokuser vores energi og ikke gabe over for meget på én gang
- Ifølge Poul er det fint ikke at have al funktionalitet færdigimplementeret, hvis man kan begrunde årsagen.

Sprint 3 - uge 21

Sprint Planning (15/05/2026) (fælles)

Sprint Goal:

- Mandag-onsdag:
 - Forsøge at færdig-implementere loginsystem
 - Hver UserRole har forskellige sider de kan tilgå efter de er logget ind
 - Det endelige loginsystem skal gerne være et samlet loginsystem for alle brugertyper
 - Oprette billeder i systemet med preview/thumbnail i frontenden
 - Måske lave forside og indsætte logo i navigationsbaren
 - Evt. se om vi kan lave det i Figma og få det over i bootstrap

- Færdiggøre ufærdige userstories
 - Sortering
 - Kalendervisning for fotoevents
 - Fotoevent tidsvalidering
- Torsdag:
 - Få opdateret diagrammer (særligt klassediagram)
 - Fortsætte rapporten
- Fredag:
 - Sprint review, Sprint retrospective og Sprint planning
 - Fortsætte rapporten

Sprint backlog og arbejdsproces (Kasper, Frederik og Maria)

I sprint 3 planlagde vi at arbejde med følgende userstories:

- Admin - sortere skoler fra en specifik kolonne
- Admin - sortere forældre fra en specifik kolonne
- Admin - sortere elever fra en specifik kolonne
- Admin - sortere skolesekretærer fra en specifik kolonne
- Admin - sortere fotografer fra en specifik kolonne
- Fotograf – uploadede billeder
- Fotograf - sortere billeder fra en specifik kolonne
- Fotograf - filtrere billeder i systemet
- Skolesekretær - uploadede oplysninger for min skole til systemet
- Skolesekretær - redigere en elev på skolens oplysninger

- Forældre - tilmelde mit barn i systemet
- Admin – oprette bookinger
- Admin –sortere bookinger fra en specifik kolonne
- Admin – bekræfte oprettelse af nye fotografer
- Admin – sortér klasselærer fra en specifik kolonne
- Admin – filtrér bookinger i systemet
- Admin – adminlogin
- Forældre – bestilling af billeder

Kodeeksempler:

Frederik:

```
42 references | Frederik Kristensen, 2 days ago | 1 author, 3 changes
public class GenericComparer<T, TKey> : IComparer<T> where TKey : IComparable
{
    private readonly Func<T, TKey> _selector;
    private readonly bool _descending;

    41 references | Frederik Kristensen, 2 days ago | 1 author, 1 change
    public GenericComparer(Func<T, TKey> selector, bool descending = false)
    {
        _selector = selector;
        _descending = descending;
    }

    0 references | Frederik Kristensen, 2 days ago | 1 author, 3 changes
    public int Compare(T? x, T? y)
    {
        if (x == null && y == null)
        {
            return 0;
        }
        if (x == null)
        {
            return -1;
        }
        if (y == null)
        {
            return 1;
        }
        int res = _selector(x).CompareTo(_selector(y));
        return _descending ? -res : res;
    }
}
```

Ovenstående kodeeksempel viser en 'GenericComparer' klasse. Den tager imod typeargumenterne T og TKey. Den implementer IComparer af den generiske type T. 'T', er

datatype som skal compares, og TKey, som forventes at implementere IComparable, er den datatype der sorteres ud fra.

Constructoren tager imod en Func, 'selector' delegate, med typeparametrene T og TKey og en bool 'descending'. Parametren 'descending' er optional, og er default værdien.

Compare metoden kalder metoden CompareTo fra vores delegate selector, hvis if statements'ne passerer, og hverken x eller y er null. CompareTo ville være individuelt implementeret i de klasser vi comparer.

Fordelen og ideen med at lave en generisk comparer, er at undgå dupliserede comparer klasser, der i virkeligheden ender med at have samme logik. Det gør det både mere maintainable samt undgår at overskride DRY-princippet¹⁵ (Don't Repeat Yourself).

Lukas:

I create photo ønskede vi at filnavnet skulle inkludere det nye photos id, så vi ikke ender med 2 ens filnavne. (2 billeder af samme elev taget samme dag fx.)

Problem: Vi kan ikke oprette et photo uden et filnavn og vi kan ikke få photoid uden at oprette et photo.

Løsning: Opret photo med tomt filnavn. Få id tilbage fra photo, så filnavn kan laves. Opdater det oprettede photo med det nye filnavn.

¹⁵ (Laursen, 2025), Prog02 s. 42

```
// Making sure FilePath is not null
NewPhoto.FilePath = "";
try
{
    NewPhoto.Photographer = await _photographerRepo.GetAsync(Convert.ToInt32(PhotographerId));
    NewPhoto.Student = await _studentRepo.GetAsync(Convert.ToInt32(StudentId));
    // We call create first to get the autogenerated identity id before we assign the filename
    await _repo.CreateAsync(NewPhoto);
    if (Photo != null)
    {
        if (Photo.FileName != null)
        {
            // Now that we have the autogenerated id we can create the correct filename
            NewPhoto.FilePath = await ProcessUploadedFile();
        }
        // The new filename needs to reach the database as well. Thus we update
        await _repo.UpdateAsync(NewPhoto);
        return RedirectToPage("Index");
    }
}
```

I onpost sætter vi NewPhoto.FilePath til en tom string for en sikkerheds skyld, så den ikke er null. Så hentes de øvrige info til photo via tidligere injection fra 2 andre repositories. Nu har vi et photo med alt undtagen et filnavn og et id.

Så kaldes CreateAsync.

```
public async Task CreateAsync(Photo item)
{
    // Added OUTPUT INSERTED.PhotoID.
    // INSERTED is the item that is inserted in the SQL.
    // OUTPUT makes it available in ExecuteScalarAsync as a return value.
    string query = "INSERT INTO Photo(FilePath,PhotoDate,PhotographerID,StudentID) OUTPUT INSERTED.PhotoID Value
    await using (SqlConnection connection = new SqlConnection(Secret.ConnectionString))
    {
        try
        {
            await connection.OpenAsync();
            SqlCommand command = new SqlCommand(query, connection);
            command.Parameters.AddWithValue("@FilePath", item.FilePath);
            command.Parameters.AddWithValue("@PhotoDate", item.Date);
            command.Parameters.AddWithValue("@PhotographerID", item.Photographer.Id);
            command.Parameters.AddWithValue("@StudentID", item.Student.Id);
            // Changed to ExecuteScalarAsync to get the outputted value.
            int newId = (int)await command.ExecuteScalarAsync();
            // The argument which is a reference type gets updated with the new Id,
            // so it is available within the model page.
            item.Id = newId;
        }
    }
}
```

Her laves et insert uden et ID. Da PhotoID er identity i vores tabel får den automatisk et nyt id. Dette skal bruges i model siden. Derfor bruges OUTPUT for at få ID ud af vores command. Der bruges ExecuteScalarAsync for at få id ud og [item.Id](#) opdateres med det nye generede id.

Da item (argumentet) er en referencetype bliver dataen i hukommelsen opdateret ét sted og når samme reference senere bruges i model siden vil de altså pege på samme objekt med det nu opdaterede id. Så mere skal ikke gøres i CreateAsync. Den behøver

ikke returnere det nye id. På den måde behøver vi heller ikke ændre i interfaces hvor CreateAsync er defineret.

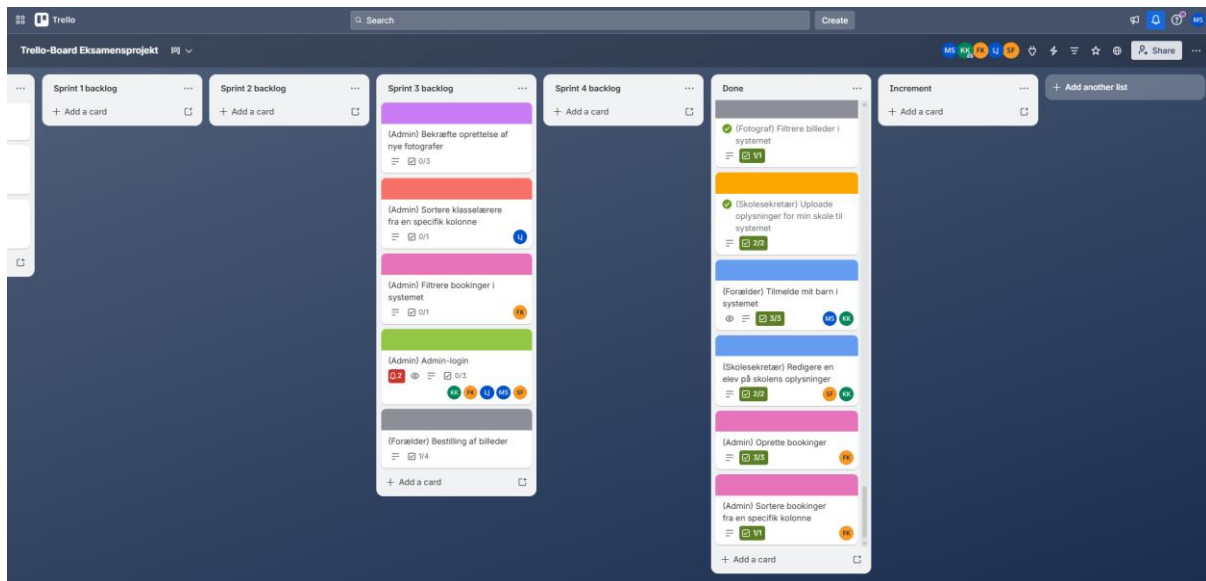
```
// The new filename needs to reach the database as well. Thus we update  
await _repo.UpdateAsync(NewPhoto);  
return RedirectToPage("Index");
```

Til sidst kaldes update async således at databasen også modtager det nye id.

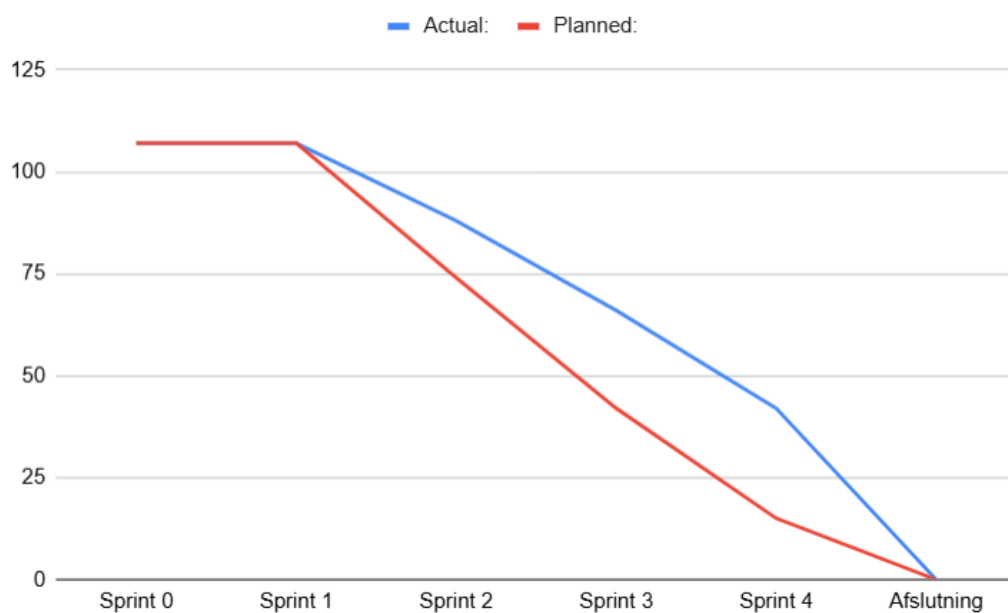
Sprint Review (22/05/2026) (Maria)

Vi har nået at lave følgende userstories (done):

- Admin - sortere skoler fra en specifik kolonne
- Admin - sortere forældre fra en specifik kolonne
- Admin - sortere elever fra en specifik kolonne
- Admin - sortere skolesekretærer fra en specifik kolonne
- Admin - sortere fotografer fra en specifik kolonne
- Fotograf – uploade billeder
- Fotograf - sortere billeder fra en specifik kolonne
- Fotograf - filtrere billeder i systemet
- Skolesekretær - uploade oplysninger for min skole til systemet
- Skolesekretær - redigere en elev på skolens oplysninger
- Forældre - tilmelde mit barn i systemet
- Admin – oprette booking
- Admin – sortere bookinger fra en specifik kolonne



Trello scrum-board slutningen af sprint 3



Burndown chart for sprint 3

Desuden gik meget af tiden i sprint 3 på rapportskrivning samt at få opdateret vores modeller og diagrammer. Vi fik også implementeret unit tests, dokumentation og regions på vores klasser. Frederik arbejdede lidt videre i sin egen branch på et login-system, men dette blev ikke færdigimplementeret, da vi vurderede at vi var nødt til at fokusere vores tid på rapporten.

Sprint 4 - uge 22

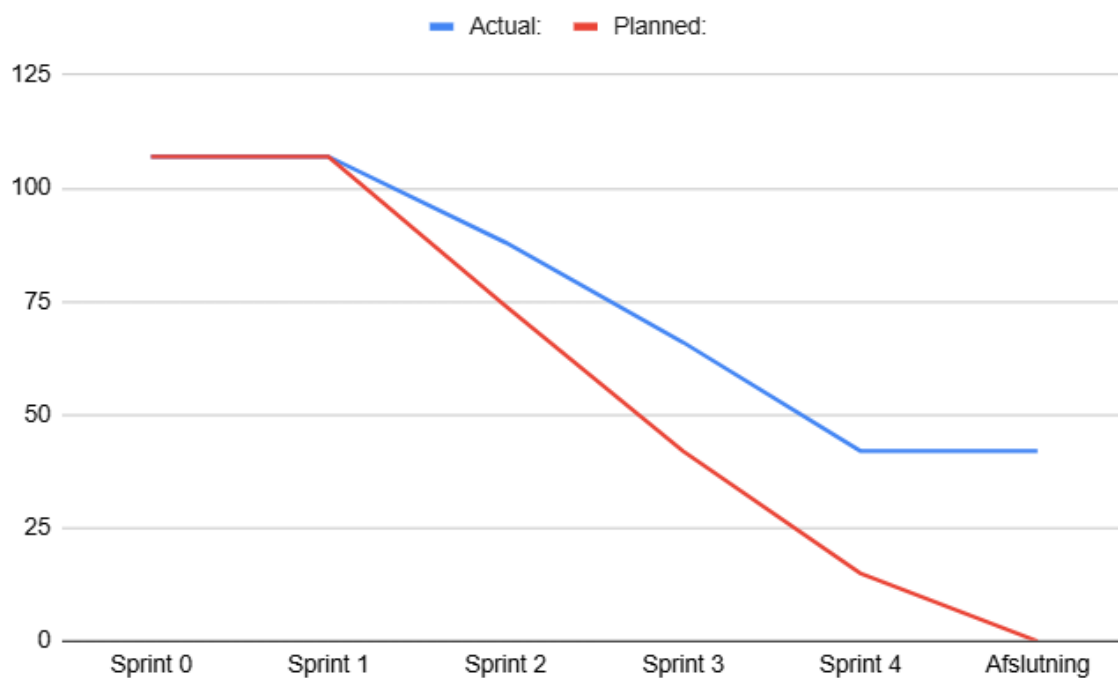
Sprint Planning (22/05/2026) (fælles)

- Weekend:
 - Unit test skal færdiggøres
 - PhotoEvent filtrering mangler og dropdown i CreatePhotoEvent (Frederik)
 - Kodeeksempel fra sprint 2 (Kasper)
 - Dokumentation og regions (hvor det mangler) SchoolClass og Student done
- Tirsdag:
 - Unit test skal færdiggøres
 - Beskrivelse af GUI og Prototype (Kasper, Sinan, Frederik og Maria)
 - **Video-præsentation (forberedelse)**
 - Beskrivelse af klassesdiagrammet skal gøres færdigt (Lukas og Sinan)
 - Show the solution forklaring (Maria)
 - Litteraturlisten skal laves
- Onsdag:
 - **Video-præsentation**
 - Refleksion over hele arbejdsprocessen
 - Simply.com upload
 - Bilag
 - Konklusion
 - Forside
 - Præsentation af system overfor Product Owner(Vicki)

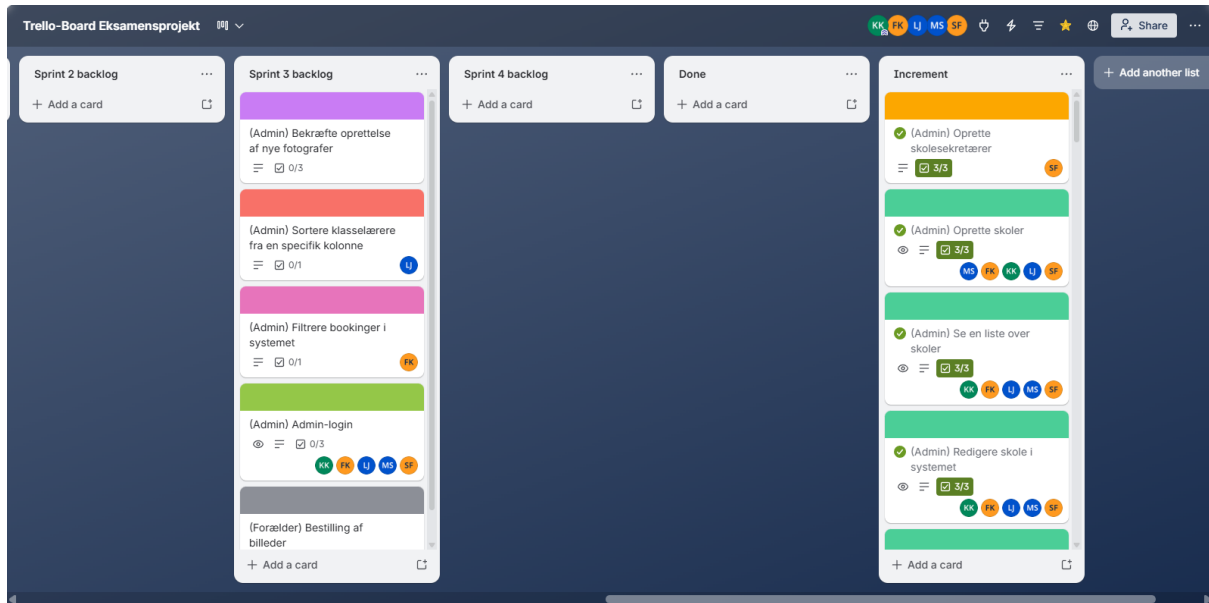
- Torsdag (**DEADLINE** 12:00) (En person opretter en gruppe med de andre og afleverer)
 - Det vi ikke har nået
 - Korrekturlæsning af hele rapporten(stavefejl, kommaer mm..)
- GUI, hvis der er tid

Sprint Review (27/05/2026) (Frederik, Maria, Kasper og Lukas)

Sprint 4 har været kendetegnet ved rapportskrivning, videofilmning og unit tests, samt færdiggjort features vi ikke havde som userstories. Dette ses afbildet på vores Burndown-chart, hvor det tydeligt fremgår, at vi ikke har arbejdet med userstories i dette sprint.



Burndown chart over sprint 4



Trello board efter sprint 4

Sprint Retrospective

Hvad fungerede godt?

- Vi har været arbejdsomme og nået meget på kort tid

Hvad fungerede mindre godt?

- Fravær og manglende kommunikation

Refleksion (diskuteret i fællesskab, skrevet af Maria)

Arbejdsproces

Vi har fundet ud af, at det betaler sig at give sig god tid til og være grundig med al forarbejdet og de indledende processer herunder oprettelse af userstories og overvejelser og beslutninger i forhold til opsætning af systemet og databasen. Dette fordi det er både tidskrævende og bøvlet at lave store strukturelle ændringer senere i forløbet.

Vi er blevet bekendt med nødvendigheden af at arbejde med userstories, men at de samtidig er enormt tidskrævende at arbejde med. Vi fandt også ud af løbende, at det

giver mest mening at opdele sine userstories i frontend og backend, da det ellers er svært at checke dem af som "Done", før frontenden er færdig.

Vi har erfaret, at det er svært at estimere arbejdsopgaver og userstories, før et projekt går i gang. Fra start er man ofte meget optimistisk omkring, hvor meget man kan nå indenfor en given tidsperiode, men det tager ofte længere tid at klare opgaver og implementere userstories, end man havde estimeret. Nogle gange går det dog også hurtigere, end man troede, og så kan man gå i gang med andre opgaver før tid og på den måde forskyde sin tidsplan.

Dette sætter streg under vigtigheden af at arbejde agilt, da faktorer meget hurtigt kan ændre sig undervejs i et projektarbejde. Der kan opstå uforudsete interne problemer i teamet, men der kan også ske ændringer fra Product Owner's side, som fx tilføjelse af ekstra userstories og funktionaliteter, som man er nødt til at få arbejdet ind i sin tidsplan. Omvendt kan der også være userstories, man er nødt til at skære fra eller rykke til en senere version, fordi man ikke kan nå dem før deadline. Det er derfor vigtigt at kunne være fleksibel, justere og omprioritere arbejdet om nødvendigt.

I forhold til skriveprocessen og rapporten har det været svært at strukturere rapporten, fordi der har været meget uklarhed om, hvordan den skulle bygges op og hvad den skulle indeholde. Samtidig blev vi undervejs i skriveprocessen flere gange gjort opmærksomme på elementer, der skulle med i rapporten, som vi ikke vidste skulle med, hvilket gjorde skriveprocessen ret stressende og kaotisk.

Samarbejde

Overordnet er samarbejdet gået godt, når alle medlemmer har været til stede. Vi har generelt været produktive og gode til at hjælpe hinanden. Dog har der været meget fravær fra især to gruppemedlemmer, og manglende kommunikation fra især ét medlem, som konsekvent er kommet for sent eller ikke dukket op. Dette har udløst en del frustration og gjort det svært at strukturere arbejdet og uddelegere opgaver, og der blev brugt unødigt meget tid på at sætte de andre medlemmer ind i, hvad vi havde lavet mens de var fraværende. Af og til har vi også arbejdet lidt i øst og vest, uden helt at vide hvad vi hver især har arbejdet med. Dette har gjort skriveprocessen ret kaotisk og

fragmenteret, og vi har manglet overblik. Dog har vores daglige Scrum møder hjulpet til at sætte en plan for dagen og snakke om, hvad vi lavede sidst. Igen er det dog kun 3 af os som har deltaget i størstedelen af disse møder, hvilket har besværliggjort arbejdet.

Konklusion (Maria, Kasper og Frederik)

Efter projektarbejdets afslutning kan vi konkludere, at vi kun til dels er kommet i mål med vores system. Vi har måtte erkende, at projektets omfang simpelthen har været for stort og omfattende til at kunne nå at blive færdige indenfor den tidsplan, vi er blevet givet. Et projekt af dette omfang tager mange måneder at strukturere og implementere, selv for professionelle og erfarne programmører. Samtidig har også udfordringer i samarbejdet gjort det svært at komme helt i mål.

Vi har dog alligevel formået at få besvaret vores problemformulering.

Vi har brugt den agile udviklingsmetode Scrum undervejs til at strukturere og dokumentere vores arbejde. Metoden har hjulpet os til først at lægge en plan for vores projekt, og undervejs til hele tiden at holde styr på vores arbejdsproces. Den agile arbejdsmetode har gjort, at vi har kunnet holde processen fleksibel, så vi løbende har kunnet justere og tilpasse vores arbejde.

Vi har brugt Business Model Generation Canvas, Inception Deck og SWOT til at analysere og få en fælles forståelse for virksomheden for bedre at kunne skabe et produkt, der tilgodeser product owners ønsker og behov.

Vi har brugt C# og en relationel database til at udvikle og implementere et mindre IT-system. Vi har benyttet Razor Pages/ASP.NET til frontend delen af vores hjemmeside, og brugt ADO.NET til vores backend, som har gjort det muligt for os at integrere en database ind i vores system, samt at vi har kunne implementere vores metoder asynkrone, som gør systemet hurtigere, da flere ting kan køre på samme tid.

Vi har brugt Git til at lave versionsstyring af koden, hostet via GitHub, så vi i fællesskab har kunne bidrage (commite) til koden samt have forskellige branches til individuelt arbejde/andre features. Desuden har vi brugt Simply.com til at hoste vores databaser, og til at publicere vores færdigudviklede prototype.

Bibliografi

- Bering, T. (November 2020). *Risikoanalyse og risikostyring*. Hentet fra CFL.dk:
<https://www.cfl.dk/artikler/risikoanalyse>
- Det skal du vide om SWOT-analyse*. (u.d.). Hentet fra Teknologisk.dk:
<https://www.teknologisk.dk/kurser/det-skal-du-vide-om-swot-analyse/46302>
- Heegaard, C. (2026). *Lær om Databaser*. Hentet fra <https://docs.google.com/>:
https://docs.google.com/document/d/1429bErDmV6MKlpyCGgS7752thmkNB81AD13V_DBNSY0/edit?tab=t.cclja69p1ghn#heading=h.s2wynlhnp96
- Laursen, P. S. (2025). *OOProg25*. Hentet fra Github:
<https://github.com/perslaursen/OOProg25>
- Maddison, V. (24. Oktober 2023). *Skolefotograferne*. Hentet fra <https://issuu.com/>:
<https://issuu.com/skolefotograferne/docs/1kzrnzyrebu>
- Nielsen, J. (30. Januar 2024). *10 Usability Heuristics for User Interface Design*. Hentet fra NN/Group: <https://www.nngroup.com/articles/ten-usability-heuristics/>
- Osterwalder, A., & Pigneur, Y. (2009). *Business Model Generation*. Amsterdam: Alexander Osterwalder & Yves Pigneur.
- Prohazka, M., & Buus, M. (u.d.). *Gestalt principper*. Hentet fra Levende Streg:
<https://levendestreg.dk/gestalt-principper/>
- Rasmusson, J. (August 2010). *The Agile Samurai*.
- Schwaber, K., & Sutherland, J. (November 2020). *Scrum Guide*. Hentet fra <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Danish.pdf>
- Skolefoto-servicesystem 2026*. (16. Marts 2026). Hentet fra docs.google.com:
https://docs.google.com/document/d/1s3XXmrHoLwb1bht_y8DWTqk8OVy_5sblFZmrEBzHN_s/edit?tab=t.0#heading=h.nf7fr03f6b21
- The Agile Inception Deck*. (6. November 2010). Hentet fra The Agile Warrior:
<https://agilewarrior.wordpress.com/2010/11/06/the-agile-inception-deck/>

Links

[Burndown-chart](#)

[Domænemodel](#)

[ER-diagram](#)

[Excel-ark med userstories](#)

[Gantt-kort](#)

[Github](#)

[Hjemmesiden](#)

[Figma](#)

[Klassediagram](#)

[Trello](#)

[Video-præsentation](#)

Bilag

Bilag 1 – Daily Scrums

Dag 1 - Projekt opstart

Info

Tilstedeværende: Lukas, Kasper, Maria, Frederik

Vejleder: MIKR

Dato: Mandag d. 16. marts 2026 (Uge 12)

Roller

- Scrum master: Lukas, Kasper, Maria, Frederik
- Referent: Lukas

Mål for i dag:

- Gruppekonspekt udfyldt
- Opsætning af dokumenter
- Virksomhedsanalyse påbegyndt
- Sprint planning af Sprint 0

Dag 2 - Interview med PO/Vicki

Info

Tilstedeværende: Kasper, Maria, Frederik, Lukas

Vejleder: MIKR OG POHE

Dato: Mandag d. 23. marts 2026 (Uge 13)

Roller

- Scrum master: Kasper, Maria, Frederik, Lukas
- Referent: Kasper, Maria

Nået sidst

- Gruppekontrakt
- Opsætning af Logbog og Rapport
- Start på Virksomhedsanalyse

Mål for i dag:

- Stille PO Spørgsmål til virksomhedsanalyse, domænet, funktionelle krav mm.
- Fortsætte med BMGC(Business model generation canvas)
- Starte med SWOT-analyse, samt inception deck

Udfordringer:

- Ingen

Dag 3 - BMGC og SWOT

Info

Tilstedeværende: Kasper, Maria, Frederik, Lukas

Vejleder: CHHE

Dato: Fredag d. 27. marts 2026 (Uge 13)

Roller

- Scrum master: Kasper, Maria, Frederik, Lukas
- Referent: Frederik, Maria og Kasper

Nået sidst

- Stille spørgsmål til PO
- Lavet videre på BMGC

Mål for i dag:

- Fortsætte med BMGC (Business model generation canvas)
- Starte med SWOT-analyse, samt inception deck

Udfordringer:

- At vi ikke har haft mulighed for at sætte Sinan ordentligt ind i vores opgave endnu.
- Lidt svært at få hjælp til ITF, da Charlotte giver individuel feedback hele dagen.

Dag 4 - Prototyper med Figma

Info

Tilstedeværende: Kasper, Maria, Frederik, Sinan og Lukas

Vejleder: MIKR

Dato: Tirsdag d. 7. april 2026 (Uge 15)

Roller

- Scrum master: Kasper, Maria, Frederik, Sinan og Lukas
- Referent: Maria

Nået sidst

- BMGC analyse færdiggjort (den skal kigges på igen på et senere tidspunkt og uddybes)
- SWOT analyse færdiggjort (den skal kigges på igen på et senere tidspunkt og uddybes)

Mål for i dag:

- Lave prototype (mock-up) med Figma, så vi kan vise Vicki de første tanker om en prototype
- Evt. start på inception deck og/eller userstories

Udfordringer:

- Vi føler, at vi mangler nogle eksempler/skærmlips af hvordan Vicki's hjemmeside ser ud nu, til inspiration og for at vide, hvilke funktioner hun har brug for

Dag 5 - Præsentation af prototyper for Vicki + spørgsmål

Info

Tilstedeværende: Kasper, Maria, Frederik, Sinan og Lukas

Vejleder: POHE

Dato: Torsdag d. 16. april 2026 (Uge 16)

Roller

- Scrum master: Kasper, Maria, Frederik, Sinan og Lukas
- Referent: Maria

Nået sidst

- Prototype færdiggjort til præsentation for Vicki
- Startet op på Inception Deck

Mål for i dag:

- Præsentere vores prototype for Vicki og få svar på afklarende spørgsmål
- Evt. arbejde lidt videre på Inception Deck?

Udfordringer:

- Vi føler, at vi mangler nogle eksempler/skærmlips af hvordan Vicki's hjemmeside ser ud nu, til inspiration og for at vide, hvilke funktioner hun har brug for
- At begrænse os - Vicki vil gerne have en masse funktionalitet, som vi ikke kan/har tid til, så vi skal fokusere på at prioritere og udvælge de ting, vi mener vi kan nå.

Dag 6 - Fortsætte arbejde med virksomhedsanalyse

Info

Tilstedeværende: Kasper, Maria, Frederik og Sinan

Vejleder: CHHE

Dato: Fredag d. 17. april 2026 (Uge 16)

Roller

- Scrum master: Kasper, Maria, Frederik og Sinan
- Referent: Kasper og Maria

Nået sidst

- Stillet Product Owner spørgsmål
- Præsenteret prototyper for Product Owner
- Enkelte tilføjelser til Inception Deck og BMGC

Mål for i dag:

- Fortsætte med virksomhedsanalyse (BMGC, SWOT og Inception Deck)

Udfordringer:

- Manglende medlemmer fra starten af dagen (Sinan) og hele dagen (Lukas)
- Generel forvirring omkring krav til virksomhedsanalysens forskellige punkter: vi får én ting at vide af Charlotte, og noget andet fra tidligere arbejdsmateriale

Dag 7 - Userstories og domænemodel

Info

Tilstedeværende: Kasper, Maria, Frederik og Lukas

Vejleder: MIKR

Dato: Mandag d. 20. april 2026 (Uge 17)

Roller

- Scrum master: Kasper, Maria, Frederik og Lukas
- Referent: Kasper og Maria

Nået sidst

- Færdig med BMGC og SWOT
- Næsten færdig med Inception deck, mangler at spørge Poul om Show your solution og NOT-list

Mål for i dag:

- Gå i gang med userstories
- Opstart på domænemodel

Udfordringer:

- Fraværende medlemmer fra dagens start

Spørgsmål:

- Behøver vi lave kataloger til alle vores klasser i domænemodellen?
- Skal vi have Admin med i vores domænemodel?
- Spørgsmål til Inception Deck
 - Show the solution

- Skal vi tænke visionært i vores Product Box, eller skal vi være realistiske i forhold til projektet? Skal vi i Meet The Neighbors skrive DAO, MobilePay med osv, eller skal de med i Not-List?

Dag 8 - Fortsat Userstories, domænemodel og estimering/burndown chart

Info

Tilstedeværende: Kasper, Maria, Frederik og Lukas

Vejleder: POHE

Dato: Onsdag d. 22. april 2026 (Uge 17)

Roller

- Scrum master: Kasper, Maria, Frederik og Lukas
- Referent: Kasper

Nået sidst

- Startede på userstories for Admin
- Startede på Domænemodel og brugte navneordsteknikken til at brainstorme

Mål for i dag:

- Få svar på vores spørgsmål til Poul
- Fortsætte på userstories og domænemodel
- Evt. opstart på estimering og burndown chart

Udfordringer:

- Fraværende medlem(Sinan)

Spørgsmål:

- ingen

Dag 9 - Fortsat Userstories, domænemodel og estimering/burndown chart

Info

Tilstedeværende: Kasper, Maria(online), Frederik og Lukas

Vejleder: POHE

Dato: Torsdag d. 23. april 2026 (Uge 17)

Roller

- Scrum master: Kasper, Maria, Frederik og Lukas
- Referent: Kasper

Nået sidst

- Arbejdede videre med userstories/domænemodel

Mål for i dag:

- Fortsætte på userstories
- Evt. opstart på estimering og burndown chart
- Starte så småt på klassediagram

Udfordringer:

-

Spørgsmål:

- ingen

Dag 10 – Gantkort Burndown chart/Estimering af userstories

Info

Tilstedeværende: Kasper, Maria, Frederik og Lukas

Vejleder: CHHE

Dato: Fredag d. 24. april 2026 (Uge 17)

Roller

- Scrum master: Kasper, Maria, Frederik og Lukas
- Referent: Maria

Nået sidst

- Arbejdede videre med userstories/domænemodel
- Opstart på klassediagram

Mål for i dag:

- Aflevere Virksomhedsanalyse til Charlotte, til gennemlæsning
- Få færdiggjort userstories
- Påbegynde estimering af userstories
- Påbegynde burndown chart
- Gantt-kort

Udfordringer:

-

Spørgsmål:

-

Dag 11 - ER- og klassediagram

Info

Tilstedeværende: Kasper, Maria, Frederik og Lukas

Vejleder: POHE

Dato: Tirsdag d. 28. april 2026 (Uge 18)

Roller

- Scrum master: Kasper, Maria, Frederik
- Referent: Frederik og Maria

Nået sidst

- Arbejdede på 1. udkast til rapport
- Estimering af userstories
- Burndown chart
- ER-diagram
- Gantt-kort
- Klassediagram

Mål for i dag:

- Videre med ER-diagram
- Videre med klassediagram
- Evt. gå i gang med at programmere (opsætning af klasser, interfaces etc.)

Udfordringer:

- Fraværende gruppemedlemmer uden besked
- Lidt uklarhed om, hvornår Sprint 1 egentlig begynder

Dag 12 - Opstart på programmering og videre med klassediagram og ER-diagram

Info

Tilstedeværende: Kasper, Maria, Lukas, Sinan og Frederik

Vejleder: POHE

Dato: Onsdag d. 29. april 2026 (Uge 18)

Roller

- Scrum master: Kasper, Maria, Frederik, Lukas, Sinan
- Referent: Frederik

Nået sidst

- Småændringer i domænemodellen
- Arbejdede videre på ER diagram
- Diskuteret og arbejdet videre på klassediagram

Mål for i dag:

- Få sat ord på ER-diagrammet og tjekke, om den er normaliseret
- Så småt gå i gang med at programmere

Udfordringer:

- Vi snakker ofte i munden på hinanden

Spørgsmål:

-

Dag 13 - Fortsætte med opsætning af modelklasser og serviceklasser

Info

Tilstedeværende: Kasper, Maria, Frederik, Sinan og Lukas

Vejleder: POHE

Dato: Torsdag d. 30. april 2026 (Uge 18)

Roller

- Scrum master: Kasper, Maria, Frederik
- Referent: Frederik, Maria og Kasper

Nået sidst

- Implementeret modelklasser i kodebasen
- Arbejdet videre og justeret klasse- og ER-diagram

Mål for i dag:

- Videre med programmering (færdiggøre modelklasserne og serviceklasserne)

Udfordringer:

- ingen

Spørgsmål:

- ingen

Dag 14 - Sprint retrospective og sprint planning

Info

Tilstedeværende: Kasper, Maria, Frederik og Sinan

Vejleder: CHHE

Dato: Fredag d. 1. Maj 2026 (Uge 18)

Roller

- Scrum master: Kasper, Maria, Frederik
- Referent: Kasper og Maria

Nået sidst

- Færdiggjort modelklasser og påbegyndt repositories
- Snak og tanker omkring opsætning af vores program og database
- Opsætning af NuGet packages
- Intro til mailopsætning

Mål for i dag:

- Sprint review og retrospective for Sprint 0
- Sprint planning af Sprint 1
- Videre med programmering, evt rettelse i klassediagram
- Rette/lave manglende userstories
- Opsætte trello til starten af sprint 1

Udfordringer:

-

Spørgsmål:

- Skal vi lave sekvensdiagrammer? Er det et krav?

Dag 15 - Påbegyndelse af sprint 1

Info

Tilstedeværende: Frederik, Kasper, Maria, Sinan (Online), Lukas (Online)

Vejleder: MIKR

Dato: Mandag d. 4 maj 2026 uge 19

Roller

- Scrum master: Kasper, Frederik og Maria
- Referent: Frederik

Nået sidst

- Sprint review og retrospective for Sprint 0
- Sprint planning af Sprint 1
- Satte klasser op i kodebasen
- Rettede/lavede manglende userstories
- Opsat trello til starten af sprint 1

Mål for i dag:

- SQL queries/Database
- Evt. start på implementering af userstories

Udfordringer:

- Discord(problemer med lyd over wifi)

Dag 16 - Fortsættelse af sprint 1

Info

Tilstedeværende: Frederik, Kasper, Maria, Sinan

Vejleder:

Dato: Tirsdag d. 5 maj 2026 uge 19

Roller

- Scrum master: Kasper, Frederik og Maria
- Referent: Kasper

Nået sidst

- Startet på implementering af userstories der omhandler skoler(Admin)

Mål for i dag:

- Fortsætte arbejde med implementering af userstories, få færdiggjort de userstories som omhandler skoler
- Fortsætte Implementering af CRUD i repositories

Udfordringer:

- ?

Spørgsmål:

- Hvordan giver vi forskellige users forskellig adgang til siderne? Skal vi have en enum 'rolle' inde i vores IUser?
- Skal vi lave regions i vores RazorPages?

Dag 17 - Fortsættelse af sprint 1

Info

Tilstedeværende: Kasper (online), Maria (online), Frederik (online), Lukas og Sinan

Vejleder:

Dato: Onsdag d. 6 maj 2026 uge 19

Roller

- Scrum master: Maria, Kasper, Frederik, Lukas
- Referent: Maria

Nået sidst

- Færdiggjort CRUD for Skole (Admin)
- Lavet SQL scripts

Mål for i dag:

- Fordele arbejdsopgaver i fht. kode
 - Maria:
 - Parent
 - Photo
 - Kasper
 - SchoolClass
 - Student
 - Frederik
 - Photographer
 - PhotoEvent
 - Sinan

- Order
- Secretary
- Lukas
- Teacher
- OrderLine

Udfordringer:

- Netværks og forbindelsesproblemer
- Strukturelle problemer med opsætning af repositories, branches, interfaces osv

Spørgsmål:

-

Dag 18 - Fortsættelse af sprint 1

Info

Tilstedeværende: Frederik, Kasper, Maria, Sinan og Lukas

Vejleder: POHE

Dato: Torsdag d. 7 maj 2026 uge 19

Roller

- Scrum master: Kasper, Frederik og Maria
- Referent: Frederik

Nået sidst

- Lavet generisk IRepository interface til alle repos

Mål for i dag:

- Fortsætte arbejde med implementering af userstories, få færdiggjort de userstories som omhandler skoler
- Fortsætte Implementering af CRUD i repositories
- Gennemgå det vi hver især har lavet siden i går

Udfordringer:

- ?

Spørgsmål:

- Skal vi lave regions i vores RazorPages?

Dag 19 - Sprint 1 review og retrospective

Info

Tilstedeværende: Frederik, Kasper, Maria, Sinan og Lukas

Vejleder: POHE

Dato: Fredag d. 8 maj 2026 uge 19

Roller

- Scrum master:
- Referent: Maria, Kasper, Frederik og Sinan

Nået sidst

- Maria: CRUD-, filtreringsmetoder og RazorPages for ParentRepositoryAsync færdiggjort. Gået i gang med PhotoRepositoryAsync
- Kasper: Oprettet sider til schoolclass og student delvist implementeret, samt implementeret repos uden at test alle metoder, nogle fungerer, nogle gør ikke
- Frederik: Implementerede CRUD funktionalitet i PhotographerRepository og lavede Razor Page's til visning, oprettelse, redigering samt sletning af fotografer..
- Sinan: Fik implementeret alle metoderne i OrderRepository

Mål for i dag:

- Få gennemgået hvad vi har lavet siden vi sidst sås, og få rettet de fejl vi hver især oplever
- Sprint review, sprint retrospective og sprint planning
- Evt. opdatere klassediagram?

Udfordringer:

- ?

Dag 20 - Sprint 2 begynder

Info

Tilstedeværende: Frederik, Kasper, Maria, Sinan

Vejleder: MIKR (online og fysisk)

Dato: Mandag d. 11 maj 2026 uge 20

Roller

- Scrum master: Maria
- Referent: Maria

Nået sidst

- Vi fik gennemgået hvad vi hver især havde lavet siden vi sidst sås
- Sprint review, sprint retrospective og sprint planning
- Opdateret Trello-board og burndown chart

Mål for i dag:

- Få kodet en del sammen i dag og rettet de fejl, som er i programmet pt
- Begynde på login funktionalitet

Udfordringer:

- Ingen kontakt til Lukas siden kl 04.00

Spørgsmål:

- Hvordan laver vi GetAll. når vi har lister af data fx. i StudentRepo med en liste af forældre? Skal vi lave om i vores database?
- Skal vi lave en separat User tabel til vores users?

Dag 21 - Sprint 2 begynder

Info

Tilstedeværende: Kasper, Maria, Frederik

Vejleder: POHE og CHHE

Dato: Tirsdag d. 12 maj 2026 uge 20

Roller

- Scrum master: Kasper
- Referent: Kasper

Nået sidst

- Forsøgt rettet fejl i systemet
- Begynde på login funktionalitet

Mål for i dag:

- Få rettet op på fejl i systemet
- Få Student til at virke
- **Få liste af parents assigned til students**
- Lave videre på loginsystem

Udfordringer:

- Manglende hjælp til løsning af fejl
- Svært at komme videre med projektet, fordi vi har mange afhængigheder, og hvis den ene ikke virker, virker den anden heller ikke
- Vi bruger enormt meget tid på at debugge og lave små ændringer i systemet

Spørgsmål:

- Hvordan laver vi GetAll. når vi har lister af data fx. i StudentRepo med en liste af forældre? Skal vi lave om i vores database?
- Skal vi lave en separat User tabel til vores users?

Dag 22 - fremvisning af systemet for Vicki

Info

Tilstedeværende: Kasper, Maria, Frederik, Sinan og Lukas?

Vejleder: POHE og CHHE

Dato: d. 13 maj 2026

Roller

- Scrum master: Frederik
- Referent: Frederik

Nået sidst

- Diskuteret implementation af loginsystem
- Ændrede at en student kan have flere forældre til kun én
- Fjernede ParentStudent tabellen
- ...

Mål for i dag:

- Få vist Vicki vores system og få feedback
- Få lavet dropdown på vores Create
- Evt. også få lavet dropdown på Edit, hvis der er tid til det
- Fikse resterende fejl

Udfordringer:

Vickis feedback:

- Eleverne mangler oplysninger - fx hvilken skole og hvilken klasse de går på
- Uploade csv filer til elever, skoler
- Tilføje mere personale til systemet
- Visning af antal elever, skoler, klasser osv.

Dag 23 - Sprint 2 review, retrospective og planlægning af sprint 3

Info

Tilstedeværende: ONLINE kl. 9:00

Vejleder: MIKR (online)

Dato: Fre 15. maj 2026 uge 20

Roller

- Scrum master: Alle
- Referent: Maria

Nået sidst

- Fået vist Vicki vores system og fået feedback
- Fået lavet dropdown på vores Create - vi fik lavet det på CreatePhoto
- Vi fik rettet op på gennemgående fejl i systemet på vores CRUD
- Tilføjet ErrorMessage for vores Required properties i vores modelklasser

Mål for i dag:

- Afslutte sprintet
- Review og retrospective
- Få opdateret userstories i vores Trello - hvilke er done?
- Få planlagt Sprint 3
- Få skrevet, HVEM der har lavet HVAD i arbejdsdokumentet

Udfordringer:

-

Spørgsmål:

-

Dag 24 - Sprint 3 start

Info

Tilstedeværende: Frederik, Kasper og Maria

Vejleder: MIKR

Dato: Man 18. maj 2026 uge 21

Roller

- Scrum master: Frederik, Kasper og Maria
- Referent:

Nået sidst

- Afsluttet sprint 2
- Sprint review og retrospective

- Vi fik opdateret vores userstories i vores Trello - hvilke skal vi arbejde med i næste sprint, og hvilke er done?
- Fik planlagt Sprint 3
- Fik skrevet, HVEM der har lavet HVAD i arbejdsdokumentet

Mål for i dag:

- Login
- Billeder i systemet
- Logo til højre for overskrift på alle sider
- Hovedside
- Implementere sortering for vores modelklasser

Udfordringer:

-

Spørgsmål:

- Hvornår skal vi starte med at gøre vores Razor page klassediagram færdigt?
- Hvilket program skal vi bruge til at lave RazorPage klassediagram i?
- Images upload, hvordan?
- SHOW THE SOLUTION?!

Dag 25 - Sprint 3 dag 2

Info

Tilstedeværende: Kasper, Frederik og Maria (Sinan og Lukas kom senere)

Vejleder: POHE

Dato: Tirs 19. maj 2026 uge 21

Roller

- Scrum master: Frederik, Kasper og Maria
- Referent: Maria

Nået sidst

- Tilføjet logo til forside
- Implementeret sortering og filtrering til modelklasser
- Forsøgt implementeret fotoupload

Mål for i dag:

- Fortsætte på login (Frederik)
- Færdigimplementeret upload af billeder (Maria, Frederik og Kasper)
- Få opdateret klassediagram og få opdateret det til RazorPage
klassediagram(Kasper og Sinan)
- Få lavet 'Show the solution' til vores Inception Deck (Maria)
- Billedeupload (fælles + Lukas)

Udfordringer:

- Manglende gruppemedlemmer
- Projektet/rapportskrivningen er uoverskuelig

Spørgsmål:

Dag 26 - Sprint 3 dag 3

Info

Tilstedeværende: Kasper, Frederik og Maria (Sinan kom senere)

Vejleder: POHE

Dato: Onsdag 20. maj 2026 uge 21

Roller

- Scrum master: Frederik, Kasper og Maria
- Referent: Frederik, Kasper og Maria

Nået sidst

- Tilføjet beskrivelser til rapporten (Maria)
- Tilføjet kodeeksempel til rapport (Lukas)
- Show the solution (Maria)
- Opdateret Trello-board med userstories, flyttet dem over i Done
- Lavet klassediagram for Models, Enums, Interfaces og Services(Kasper og Sinan)
- Lavet videre på loginimplementation (Frederik)
- Færdigimplementeret imageupload

Mål for i dag:

- Fortsætte på login (Frederik)
- Tilføj comparer kodeeksempel til rapport (Frederik)
- Fortsætte klassediagram
- Unit test
- ER-diagram?
- Evt. start på litteraturliste

- Dokumentation og regions

Udfordringer:

- Sinan forsinket
- Ikke hørt fra Lukas

Spørgsmål:

Dag 27 - Sprint 3 dag 4

Info

Tilstedeværende: Kasper, Frederik, Maria, Lukas og Sinan (online)

Vejleder: POHE

Dato: Tors. 21. maj 2026 uge 21

Roller

- Scrum master: Frederik, Kasper og Maria
- Referent: Frederik, Kasper og Maria

Nået sidst

- Arbejdet videre med login (Frederik)
- Skrevet videre på rapport (Maria)
- RazorPage klassediagram - få relationer mangler
- Opdateret Trello
- Implementeret placeholder-billede til vores mockdata i databasen
- ER-diagram opdateret
- Domænemodel opdateret

- Unit-test (Lukas)
- Regions på SchoolClass og Student (Kasper)

Mål for i dag:

- Tilføj comparer kodeeksempel til rapport (Frederik)
- Færdiggøre klassesdiagram med relationer
- Unit test
- Evt. start på litteraturliste
- Dokumentation og regions
- Fortsætte på rapportskrivning

Udfordringer:

-

Spørgsmål:

Dag 28 - Sprint 3 dag 5

Info

Tilstedeværende:

Vejleder: Ingen

Dato: Fre. 22. maj 2026 uge 21

Roller

- Scrum master: Frederik, Kasper og Maria
- Referent:

Nået sidst

- Tilføjet comparer kodeeksempel til rapport (Frederik)
- Færdiggjort klassesdiagram med relationer
- Unit test (Lukas)
- Vi startede på litteraturlisten
- Dokumentation og regions (Sinan og Frederik)
- Fortsætte på rapportskrivning
- Burndownchart for sprint 3

Mål for i dag:

- Unit testing for andre repos
- Fortsætte dokumentation og regions
- Fortsætte rapportskrivning
- Afslutning af sprint 3 med review og retrospective
- Planlægning af næste sprint

Udfordringer:

-

Spørgsmål:

- Skal vi skrive i koden, hvem der har lavet hvad?
- Må vi have et refleksionsafsnit over hele vores opgave?
- Hvad skal man præsentere i videoen?
- KILDER - skal vi skrive kilde på, hvis vi nævner 'DRY' fx?

Dag 29 - Sprint 4 sidste uges arbejde starter

Info

Tilstedeværende: Sinan, Frederik, Maria og Kasper

Vejleder: Ingen

Dato: Tirs. 26. maj 2026 uge 22

Roller

- Scrum master: Sinan, Frederik, Kasper og Maria
- Referent:

Nået sidst

- Unit testing for andre repos
- Dokumentation og regions(på nogle klasser)
- Rapportskrivning
- Review og retrospective
- Planlægning af sidste sprint

Mål for i dag:

- Beskrivelse af GUI og Prototype (Sinan og Frederik)
- Show the solution forklaring (Maria)
- Beskrivelse af klassediagrammet skal gøres færdigt (Lukas)
- Unit test skal færdiggøres
- Litteraturlisten skal laves
- **Video-præsentation (forberedelse)**

Udfordringer:

- Lukas dukkede ikke op

Spørgsmål:

- Skal vi skrive i koden, hvem der har lavet hvad?
- Må vi have et refleksionsafsnit over hele vores opgave?
- Hvad skal man præsentere i videoen? Poul er blevet spurgt og det er kun frontend
- KILDER - skal vi skrive kilde på, hvis vi nævner 'DRY' fx?

Dag 30 - Sprint 4 - det sidste

Info

Tilstedeværende: Sinan, Frederik, Maria og Kasper

Vejleder: POHE

Dato: Ons. 27. maj 2026 uge 22

Roller

- Scrum master: Sinan, Frederik, Kasper og Maria
- Referent:

Nået sidst

- Færdiggjort unit test for parent og photographer
- Færdiggjort dropdown selection i photographer og photoEvent
- Skrevet videre på rapport

Mål for i dag:

- Præsentere for Vicki
- Unit test skal færdiggøres (Sinan)
- Requirement og independent injection kode dokumentation (Sinan)

- Video?????
- Litteraturlisten
- Så færdiggjort sprint 4
- Bilag?
- Simply upload
- Korrekturlæsning
- Forside på rapporten
- Forside på hjemmesiden

Udfordringer:

-

Spørgsmål:

- Skal vi skrive i koden, hvem der har lavet hvad?
- Må vi have et refleksionsafsnit over hele vores opgave?
- Hvad skal man præsentere i videoen? Poul er blevet spurgt og det er kun frontend
- KILDER - skal vi skrive kilde på, hvis vi nævner 'DRY' fx?
- Må man commit'te efter afleveringfristen? Hvis vi fx laver et login eller en forside

Feedback fra Vicki på præsentationsdagen

- Bliver password opbevaret?
- Man kunne godt have lavet en funktionalitet, der gør at man kan se passwordet, når man skriver det, så man er sikker på, man ikke har skrevet forkert
- Man kunne også lave et 'gentag password' felt
- Skoler

- Mangler individuelle sider - fx at man i listen over skoler kan klikke ind på skolen og se flere detaljer
 - Antal elever
 - Antal klasser
 - Datoer og klokkeslæt over fotograferinger
 - Kontaktinfo (kontaktperson)
 - Vindue med noter - fx til sekretæren
- Skoleklasser
 - Forældre kan ikke huske årgang
 - Løsningen Vicki kom op med var at klasserne automatisk fik nuværende år (2026) som årgang
- Skolesekretær og lærere
 - Initialer mangler
 - Stilling mangler

Bilag 2 – Elevator Pitch skabelon



The Elevator Pitch

- For [individual construction teams]
- who [need track road access on the construction site],
- the [Road Closure System (RCS)]
- is a [safety communication tool],
- that [informs crews when roads will be closed].
- Unlike [the current paper-based system]
- our product [is web based and can be accessed
- by all contractors anywhere anytime].

(The Agile Inception Deck, 2010)

Bilag 3

	Sprint 0							Sprint 1	Sprint 2	Sprint 3	Sprint 4
	Uge 12	Uge 13	Uge 14	Uge 15	Uge 16	Uge 17	Uge 18	Uge 19	Uge 20	Uge 21	Uge 22
Virksomhedsanalyse											
Prototyper											
User Stories + estimering											
Domænemodel og klassediagram											
Burndown Chart											
ER-diagram											
Sekvensdiagram											
Implementering (programmering)											
UnitTest											
BrugerTest											
Evt. justering											
Deadline											28-05-2026

Bilag 4 – Scrum Event planlægning

Scrum Events

Sprints varighed:

Sprint 0: 6 uger (kun hver mandag/tirsdag)

Sprint 1: 1 uge

Sprint 2: 1 uge

Sprint 3: 1 uge

Sprint 4: 1 uge

Sprint Planning:

Afholdes før hvert sprint om fredagen for at få afklaring på hvilke ting vi skal nå i hvert sprint.

Daily Scrum:

Vi har tænkt os at holde daily scrum hver morgen fra kl. 9:15-9.30

Sprint Review:

Efter hvert sprint, for at sættes os sammen og se hvad scrum-teamet har lavet i det tidligere sprint. Hver fredag efter hvert sprint.

Sprint Retrospective:

Refleksioner over nuværende sprint

Scrum Events

Sprints varighed:

Sprint 0: 6 uger (kun hver mandag/tirsdag)

Sprint 1: 1 uge

Sprint 2: 1 uge

Sprint 3: 1 uge

Sprint 4: 1 uge

Sprint Planning:

Afholdes før hvert sprint om fredagen for at få afklaring på hvilke ting vi skal nå i hvert sprint.

Daily Scrum:

Vi har tænkt os at holde daily scrum hver morgen fra kl. 9:15-9.30

Sprint Review:

Efter hvert sprint, for at sættes os sammen og se hvad scrum-teamet har lavet i det tidligere sprint. Hver fredag efter hvert sprint.

Sprint Retrospective:

Refleksioner over nuværende sprint